

Bloodhound Auxiliary Sensors



Prepared by: Rudy Earnest, Jonas Cram

Robotics and Communications Systems Engineering Technology, Idaho

State University

Prepared for: Shane Slack, Tim Rossiter

Spring 2026

Table of Contents

Table of Figures and Tables.....	4
1: Abstract.....	6
2: Background.....	7
3: Proposed Problem.....	8
3.1: Overview.....	8
3.2: Semester Goals.....	8
4: Environmental & Ethical Considerations.....	9
5: System Overview.....	12
6: Stage Overview.....	14
6.1: Camera Slave.....	14
6.1.1: Camera Commands.....	14
6.1.2: Camera Slave Program.....	18
6.1.3: Camera Slave Circuit REV A.....	28
6.1.4: Buck Converter.....	34
6.1.5: VB Remote Testing TX Mode Form.....	36
6.2: LiDAR - Light Detection and Ranging.....	38
6.2.1: LiDAR Packet.....	38
6.2.2: Decoding the Data Packet.....	40
7: Troubleshooting Procedure.....	45
7.1: Camera Slave Troubleshooting.....	45
7.1.1: Power Verification.....	45
7.1.1.1: Buck/Regulator Output Input.....	45
7.1.1.2: +5V Linear Regulator Output (TP4).....	46
7.1.1.3: -5V Buck Inverter (TP7).....	47
7.1.2: Digital/Control Signal Verification.....	48
7.1.2.1: I2C Bus from Master/RX Controller (TP2 & TP3).....	48
7.1.2.2: Slave UART Output (TP5 & TP6).....	49
7.1.2.3: Camera Address Bits (TP8 & TP9).....	50
7.1.2.4: Camera output header.....	51
7.1.3: Video Signal Troubleshooting.....	51
7.1.3.1: Confirm Camera Address (TP8 & TP9).....	51
7.1.3.2: Board Video Signal (TP10).....	51
7.1.3.3: RF Transmission Verification.....	52
8: Cost Analysis.....	54
9: Conclusion & Recommendations.....	55
10: Appendix.....	59

10.1: 3D models.....	59
10.2: BOM.....	62
10.2: Engineering Logs.....	66
10.3: PCB.....	68
10.4 Flowcharts.....	72
10.4: Github.....	74
10.5: Schematics.....	75
11: References.....	77

Table of Figures and Tables

Figure 5.1: System Block Diagram.....	12
Figure 6.1: Master Byte Packets.....	14
Table 6.1: Button Layout.....	15
Figure 6.2: Example UART Transmission.....	16
Table 6.2: Command Details.....	17
Figure 6.3: Main Program Flowchart.....	18
Figure 6.4: ISR Flowchart.....	20
Figure 6.5: Translate Y Flowchart.....	22
Figure 6.6: Translate X Flowchart.....	23
Figure 6.7: Translate TABXY Flowchart.....	24
Figure 6.8: Translate J Buttons Flowchart.....	26
Figure 6.9: Lookup and Send Command Flowchart.....	26
Figure 6.10: Power Schematic.....	28
Figure 6.11: Digital Circuitry Schematic.....	30
Figure 6.12: Outputs Schematic.....	32
Figure 6.13: REV A PCB layout.....	33
Figure 6.14: Buck Schematic.....	34
Figure 6.15: Fan Buck Case.....	35
Figure 6.16 Universal Remote Testing TX Mode.....	36
Figure 6.17: Universal Remote Testing RX mode.....	36
Figure 6.18: Raw LiDAR Data Waveform.....	38
Figure 6.19: LiDAR TTL/RS-232 Interface Circuit.....	41
Table 6.3: Formatted/Indexed LiDAR Data.....	43
Figure 6.20: C# GUI - Data Translation Attempt Output.....	44
Figure 7.1: Expected Buck/Regulator Voltage.....	45
Figure 7.2: TP4 Expected Voltage.....	46
Figure 7.3: Expected TP7 Voltage.....	47
Figure 7.4: Expected TP2 and TP3 Waveform.....	48
Figure 7.5: Expected TP5 and TP6 Waveform.....	49
Table 7.1: Camera Address Bits.....	50
Figure 7.6: TP8 and TP9 Camera 2 Selected.....	50
Figure 7.7: Expected TP10 Waveform.....	51
Figure 7.8: Example Spectrum Analysis.....	52
Figure 10.1: 90mm Fan Buck Case Sketch.....	59
Figure 10.2: Front View.....	60

Figure 10.3: Side View.....	60
Figure 10.4: Top View.....	61
Table 10.1: Project BOM.....	65
Table 10.2: Rudy's Time Log.....	66
Table 10.3: Jonas's Time Log.....	67
Figure 10.5: Camera PCB Front Copper.....	68
Figure 10.6: Back Copper.....	69
Figure 10.7: Front Silkscreen.....	70
Figure 10.8: Back Silkscreen.....	71
Figure 10.9: Camera Slave Program Overview Flowchart.....	72
Figure 10.10: C# UART LiDAR Program Overview Flowchart.....	73

1: Abstract

The Bloodhound auxiliary project supports Hood Tech cameras and other 13V-compatible cameras, as well as laying the groundwork for LiDAR in the future. The camera control slave enables video feed selection across all connected cameras, while pan-tilt-zoom (PTZ) control and additional advanced functions are exclusive to Hood Tech cameras. This will be used on project Bloodhound for control of the cameras on this search and rescue robot. There are two main components for the camera project, the universal controller and the camera control slave board. The camera control slave board receives information from the universal controller master, then gets converted into commands to control the Hoodtech cameras then is sent to the selected camera. The video feed of the selected camera is transmitted back to the user enabling them to have visual control over the robot and its actions. The LiDAR system currently can read the data off of the LiDAR module, but requires further development in the future to have full usability within project Bloodhound.

2: Background

The hood tech camera control project was initially started by Carson Bogart and Dane Davids in the Spring of 2023. The cameras were originally donated in 2016 and others such as Chuck Kibbie and Tristan Worley worked with the cameras as well. Provided at the start of this year was the original test program for the cameras, the connection diagram, and a VB program previously developed to control the camera.

The LiDAR module integration was started this semester (Spring 2026) and has no previous documentation for implementation in other projects.

3: Proposed Problem

3.1: Overview

The previous semester's work used a Visual Basic program on a computer to issue commands to the Hood Tech cameras. This approach, while effective in a lab setting, is ineffective with robotic deployment where independence from a tethered computer is a critical factor. Therefore, a new system must be developed for remote robotic operations. This system should integrate other sensors along with a wireless controller already developed for robotic control applications, as well as wirelessly transmit the video back to the remote operator. Furthermore, this system should be able to be used by any other system seeking to integrate the Hood Tech cameras in the future.

3.2: Semester Goals

- Develop I2C protocol for camera control
- Develop multi-cam use circuit
- Develop recognition for camera and integrate other sensors
- Refine Camera interface
- Develop TX and RX for camera and sensor data

4: Environmental & Ethical Considerations

When beginning a new project, the impacts of technology and ethical concerns must be considered in accordance with social, economic, environmental, safety, and social implications. This project pertaining to interfacing with a thermal camera, then being utilized for search and rescue purposes must be analyzed within these contexts as well. While the project at its core has no malicious intent, it still has potential for misuse and damaging environmental concerns.

The main benefit that thermal imaging provides in a search and rescue (SAR) scenario is the improvement in survival rates during time sensitive missions. Research done by Ulloa et al. indicates that these in combination with the robotic systems that they are attached to achieve a high degree of accuracy of identifying victims via infrared (IR) radiation detection. This allows them to “detect people in various positions and even when they are partially covered” (Ulloa et al.). This then provides a safety buffer between the dangerous situation and the human rescuers by sending in a robot first to navigate the treacherous situation and find any human within this area of danger. Economically, this reduces the strain on an emergency department’s budget by theoretically shortening the window for search and minimizing risk of injuries and/or fatalities for first responders.

Despite the clear safety advantages of an SAR situation, the social impacts of thermal imaging are highly disputed, particularly regarding privacy concerns. Some argue that public fears regarding robot surveillance

are often based on “misconceptions of privacy.” Naomi Lintvedt suggests that the way we perceive privacy when it comes to a robot differs from how we perceive it with a human or a fixed camera. The context of the interaction, such as a lifesaving situation, can alter the ethical weight of the data collection (Lintvedt). From this perspective, the “intrusiveness” of a thermal imaging sensor is a necessary tradeoff for human detection required to save someone’s life.

Conversely, the Electronic Frontier Foundation (EFF) offers a different viewpoint, arguing that thermal imaging represents a “dangerous dragnet” for surveillance. The EFF states that these cameras do not just “see” heat, rather, they record biological data that can be used for over policing or tracking individuals through facial recognition. This highlights an area of mission creep where a tool designed for SAR purposes could be repurposed for surveillance compromising civil liberties.

Beyond a social debate, the physical construction of this project carries some environmental concerns. The use of batteries and PCBs introduces the risk of polluting the environment with e-waste. If this robot gets lost in a disaster zone or enters a mission with temperatures that the robot is not capable or rated to handle, these components can become long-term pollutants. This can also lead to a worsening situation in the case of a battery fire, counteractively increasing the risk of the already dangerous situation that the robot was intended to help mitigate.

Thermal-equipped SAR robots are valuable tools for modern EMS strategies, offering a high accuracy method for locating survivors while mitigating the potential risk for the loss of human life. However, the technology is not without privacy concerns within its collection of data. Moving forward, the balance between privacy, safety and environmental impact must be balanced between both implementing strict management of data collected and choosing sustainable methods for construction of materials wherever possible. By addressing these concerns at the core of the project, the technology can be utilized without compromising values and the environment that must be protected.

5: System Overview

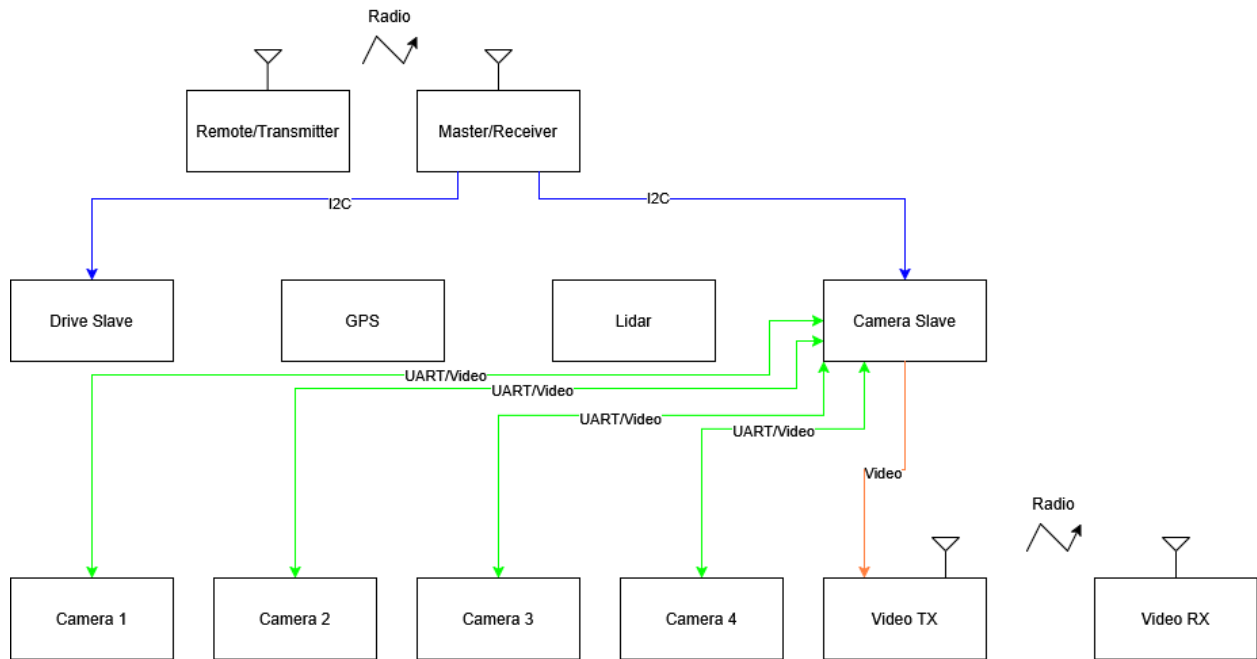


Figure 5.1: System Block Diagram

The parts of this system that will be covered within this report are the camera slave and the LiDAR. The other parts, GPS and drive slave, are covered by another report that is working on the “Bloodhound” project as well during this same semester.

The function of the Camera Slave relies on the universal remote and master that was created previously for robotic control applications. The combination of these two devices transmit the information of the joysticks and various other buttons pressed on the controller to the master board via radio transmission. The transmitted data is then received by the masterboard, which then builds the information into byte packets to be sent over I2C. These I2C byte packets are directed to the camera slave and the

drive slave boards. The camera slave converts the I2C commands into byte packets that the Hood Tech cameras can understand, then passed to the selected camera via UART. The video signal from the connected cameras is sent to the camera slave. The video feed of the selected camera then gets transmitted over radio to be received by a video capture receiver.

The functioning LiDAR module would only be reliant on a slave board that would be used to translate and process the received raw UART data into relevant/usable information regarding the angle sector of the scan (directionality), coupled with the measured distance of a detected object. If an object was detected to be too close, an I2C signal from the LiDAR's slave board would be sent to alert the master of the detected close proximity.

6: Stage Overview

6.1: Camera Slave

6.1.1: Camera Commands

The camera slave waits for a byte packet from the receiver/master board over I2C. The packet string sent from the master can be seen in the figure below.

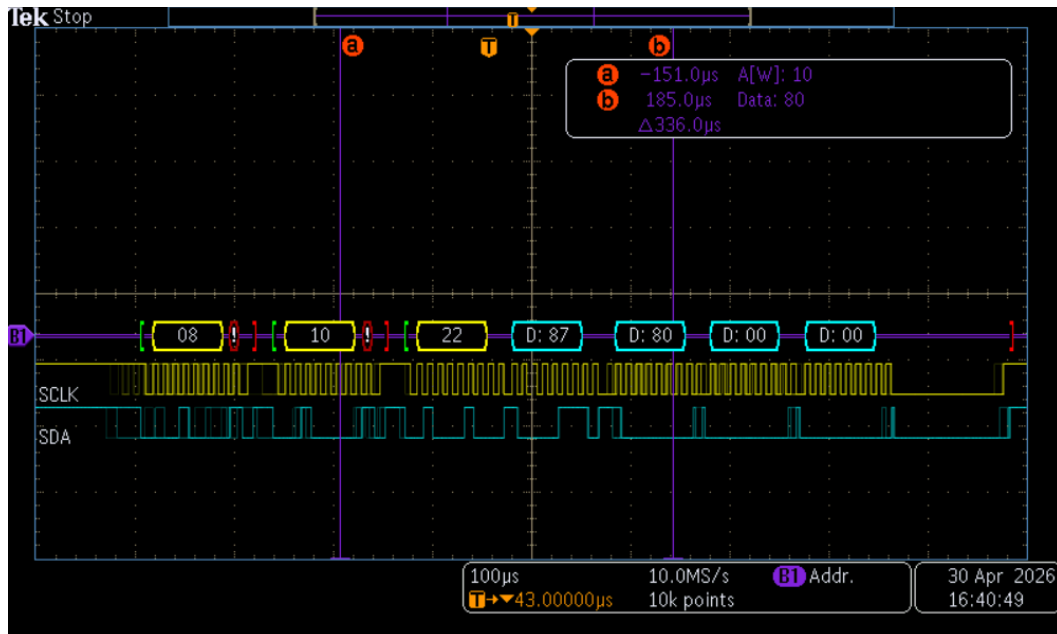


Figure 6.1: Master Byte Packets

Button/Joystick Move	Byte Change from Master	Command Sent to camera
Joy 3 Down	2nd Byte = 0x00 to 0x32	5 degree tilt down
Joy 3 Down	2nd Byte = 0x33 to 0x5A	1 degree tilt down
Joy 3 Down	2nd Byte = 0x5B to 0x7B	.2 degree tilt down
Joy 3 Up	2nd Byte = 0x95 to 0xB5	.2 degree tilt up
Joy 3 Up	2nd Byte = 0xB6 to 0xDD	1 degree tilt up
Joy 3 Up	2nd Byte > 0xDD	5 degree tilt up
Joy 3 Right	3rd Byte = 0x00 to 0x32	5 degree pan right
Joy 3 Right	3rd Byte = 0x33 to 0x5A	1 degree pan right
Joy 3 Right	3rd Byte = 0x5B to 0x7B	.2 degree pan right
Joy 3 Left	3rd Byte = 0x95 to 0xB5	.2 degree pan left
Joy 3 Left	3rd Byte = 0xB6 to 0xDD	1 degree pan left
Joy 3 Left	3rd Byte > 0xDD	5 degree pan left
Joy 3 button	5th Byte 0x01	Home
A	4th Byte 0x01	Reset
B	4th Byte 0x02	Video On
X	4th Byte 0x04	Not sent to camera, switches mux outputs for video and transmission
Y	4th Byte 0x08	FFC/Adjust
LT	4th Byte 0x10	Zoom out
RT	4th Byte 0x20	Zoom in

Table 6.1: Button Layout

Table 6.1 details the buttons that can be pressed or the joystick movements that can be done that affect the camera. When looking at the second column refer to Figure 6.1 starting with the first byte of the 0x22 address for which byte is being referenced. The ranges given in the second column for left right up down indicate that when the byte is within this range this is what will happen. A gap in the ranges might have been noticed, this is because when it is within 0x7C to 0x94 there is no movement command in any direction. Also note that when sending a diagonal input the Y (vertical) direction sends before the X (horizontal) direction. What this means is that as of right now the X direction overrides the Y directions command resulting in the camera either “stuttering” with a diagonal joystick input or the camera will move a majority in the X directions (Left, Right). Table 6.2 details the byte packets sent to the camera in reference to Table 6.1’s command names. The commands to the camera are sent out at a 57600 baud rate and an example can be seen in Figure 6.2.

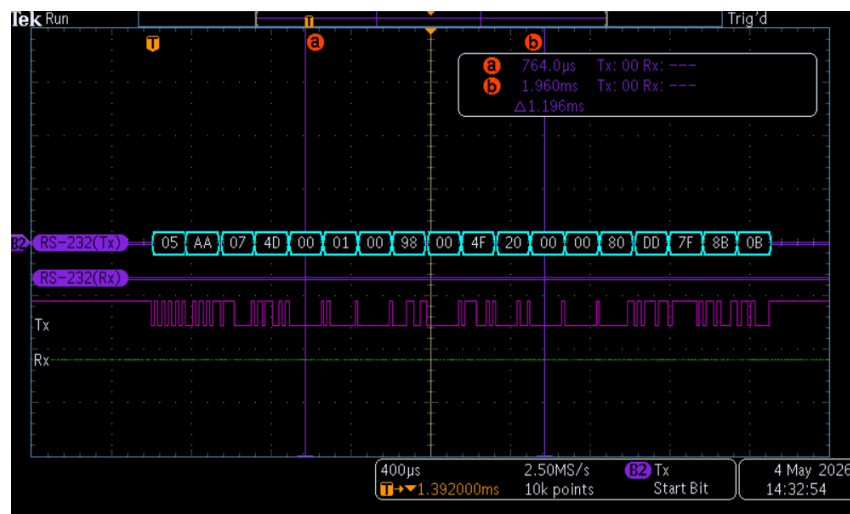


Figure 6.2: Example UART Transmission

Command	Bytes Sent
Up .2 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, DD, 7F, 8B, 0B
Up 1 degree	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, 51, 7F, 4B, 6F
Up 5 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, 97, 7C, EA, 7C
Down .2 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, 23, 80, AB, 0B
Down 1 degree	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, AF, 80, 6B, 6F
Down 5 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 00, 80, 69, 83, CA, 7C
Left .2 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, DD, 7F, 00, 80, C7, 18
Left 1 degree	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 51, 7F, 00, 80, 57, 32
Left 5 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 97, 7C, 00, 80, DF, F3
Right .2 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 23, 80, 00, 80, 1F, 19
Right 1 degree	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, AF, 80, 00, 80, 8F, 33
Right 5 degrees	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 20, 00, 69, 83, 00, 80, 07, FF
Zoom in	55, AA, 07, 4D, 00, 01, 00, 98, 00, 5A, 01, 00, 00, 00, 01, 00, F3, D8
Zoom out	55, AA, 07, 4D, 00, 01, 00, 98, 00, 5A, FF, 00, 00, 00, 01, 00, 2D, CD
Zoom stop	55, AA, 07, 4D, 00, 01, 00, 98, 00, 5A, 00, 00, 00, 00, 00, 00, B2, DB
Video On	55, AA, 07, 4D, 00, 01, 00, 98, 00, 7C, 00, 00, 00, 00, 00, 00, 70, 9F
Home	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, 4A, 19, B8, 8B, 00, 80, 31, 53
Reset Camera	55, AA, 07, 4D, 00, 01, 00, 98, 00, 47, FE, 00, 00, 00, 00, 00, 6D, 00
FFC/Adjust	55, AA, 07, 4D, 00, 01, 00, 98, 00, 6D, 01, 03, 00, 00, 00, 00, A0, DB

Table 6.2: Command Details

6.1.2: Camera Slave Program

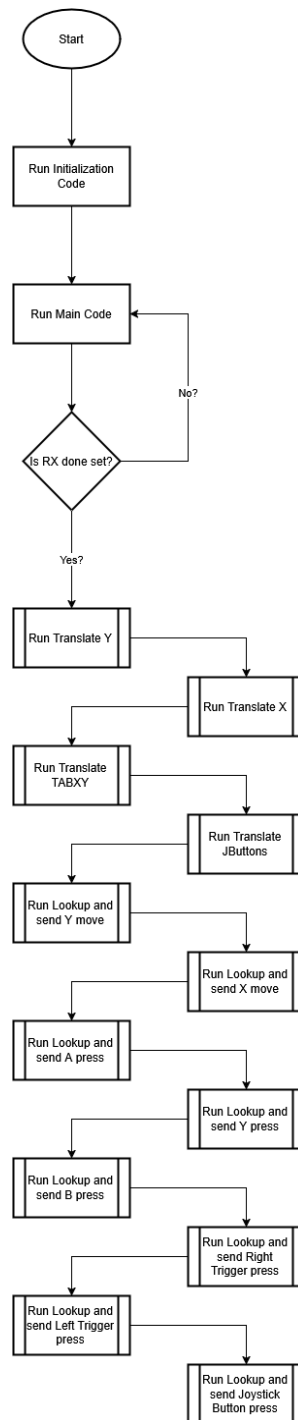


Figure 6.3: Main Program Flowchart

The camera's Main program, shown in Figure 6.3, waits for the boolean to be set by the Interrupt-Service Routine (ISR) to process and send the bytes out to the camera. The order that this is done in follows the order in Figure 6.3. Once all of the bytes have been translated into the commands that are going to be sent to the camera, the “lookup and send” process is called for each byte and depending on if it is a button or a movement a different “lookup and send” process is called. These commands are transmitted from the UART TX pin to the selected camera.

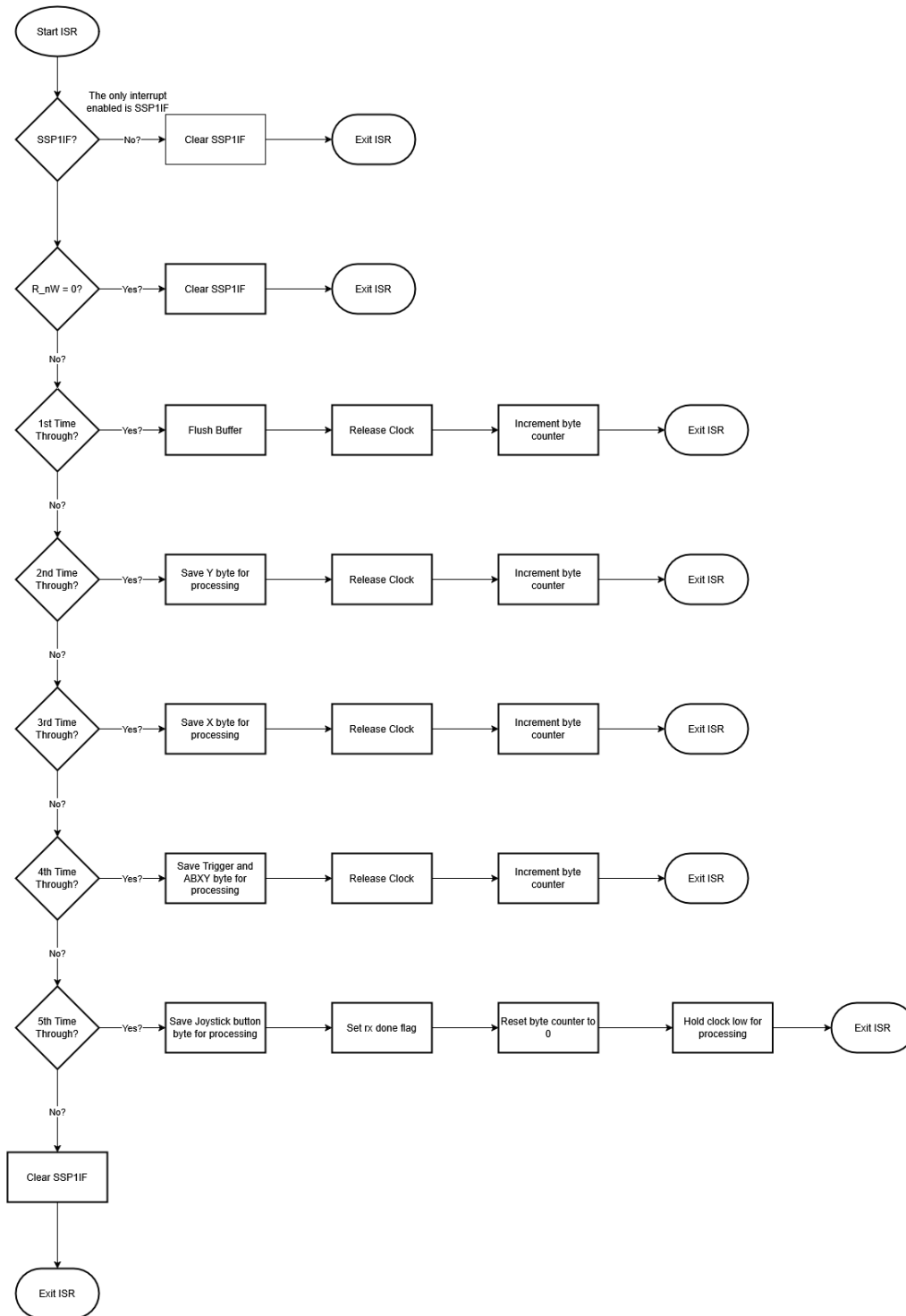


Figure 6.4: ISR Flowchart

The ISR flowchart, shown in Figure 6.4, is based on which byte of the packet sent from the Master you are currently on. Since the byte structure is set and will never change, the program will always receive and handle the bytes in the same order. The first byte is ignored and removed if the address value is correctly matched to the slave. The next byte is the position data for the Y potentiometer of the joystick, which is stored for later processing. Next, the X byte, the triggers and ABXY buttons byte and the byte containing the joystick buttons gets stored. After each individual byte has been stored, the I2C clock is released by the slave and the byte counter variable gets incremented, which tracks the number of received bytes. After the 5th byte (joystick byte), the clock remains held low to allow processing to take place as well as allowing the camera commands to be sent without interruption. After all of this takes place, the byte counter is reset back to zero.

The clock being held low may cause issues with the drive of the Bloodhound because of the longer processing and transmission time required, especially if multiple commands are used. Full system testing will need to take place to verify and correct this possible issue.

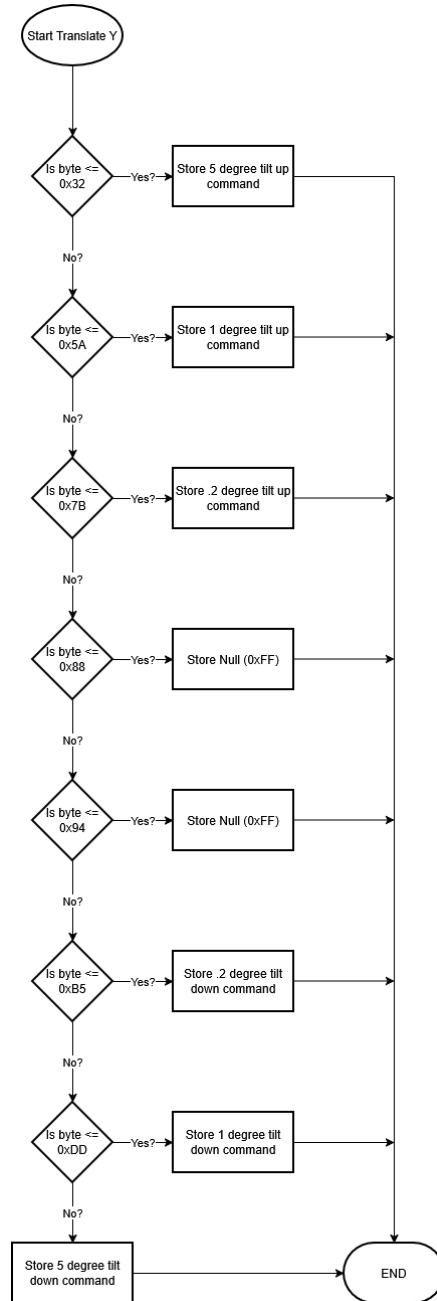


Figure 6.5: Translate Y Flowchart

The translate Y process, shown in Figure 6.5, takes the Y byte received from the master and reads the value of the sent ADC value. If the stick is pushed further, the byte value increases or decreases, respectively, causing the movement of the pan/tilt movements to increase proportionally as well.

If the stick is at the home position or “dead zone,” a null value is placed within the register. This dead zone value should be 0x7C to 0x94. In order for the camera to pan/tilt, the ADC value must be outside of the bounds of the dead zone.

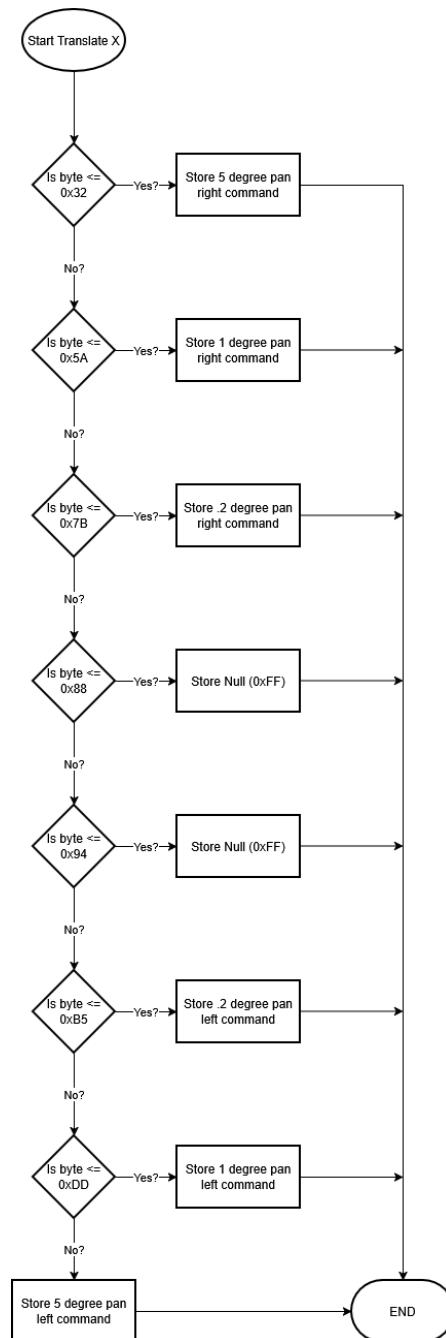


Figure 6.6: Translate X Flowchart

The translate X process shown in Figure 6.6 follows the same process as the translate Y function. This time however, the commands are stored in accordance with the pan left and right commands for the “lookup and send” process.

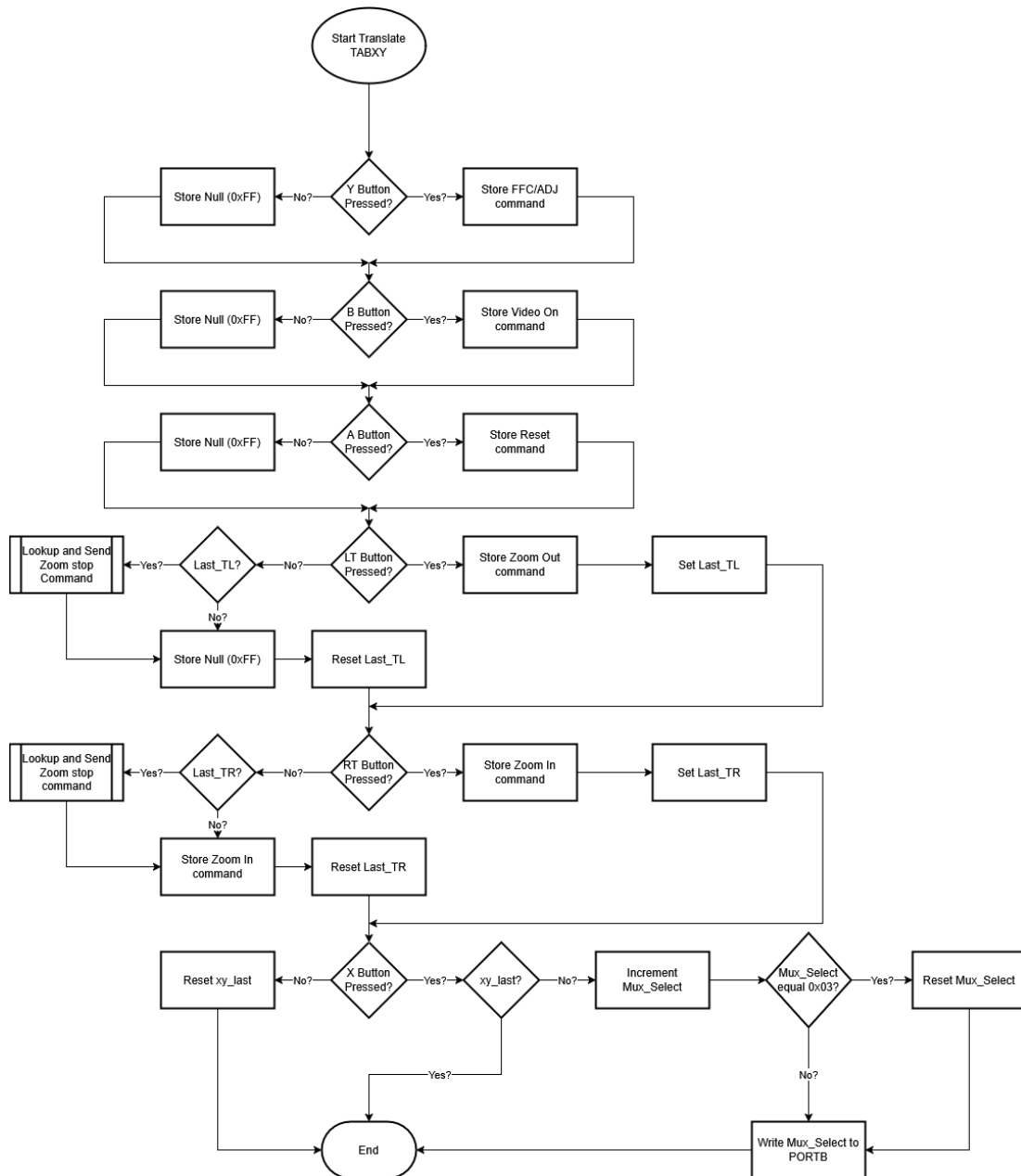


Figure 6.7: Translate TABXY Flowchart

The translate TABXY process shown in Figure 6.7 checks each button and trigger to see if they are pressed. If the A, B, or Y buttons have been pressed, the command for that button's position in the lookup table is stored within the respective register. If the button has NOT been pressed, a null value (0xFF) is stored to later tell the “lookup and send” process to not send a command.

If a trigger has been pressed, the correlating lookup table position is stored in the register and the respective “this button was pressed last cycle” boolean flag is set. When the trigger is NOT pressed, it checks to see if the boolean flag was set. If the boolean flag WAS set, the program sends the zoom stop command to prevent the camera from continuing to zoom, then places a null value in the register. If the boolean flag was NOT set, a null is placed in the register.

If the X button is pressed, the program checks if it was pressed last cycle. If it WAS pressed last cycle, then PORTB remains unchanged, otherwise PORTB is incremented up to three before being reset back to zero making for four total positions. When the button is NOT pressed, the boolean flag is reset to FALSE (0), ensuring that the X button has to be released and pressed again to toggle to the next camera, providing a predictable “step” when changing between cameras..

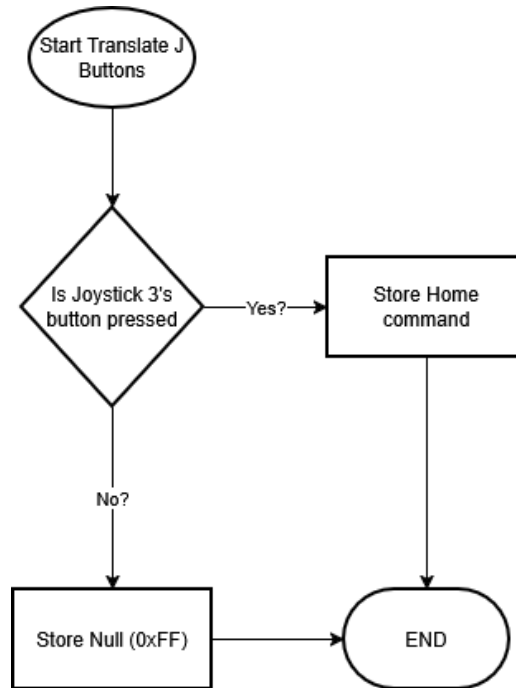


Figure 6.8: Translate J Buttons Flowchart

The Translate J Buttons' process, shown above, tests to see if joystick 3's button has been pressed; If it has been, the lookup table value for the home command is stored in a register. If it hasn't a null is placed instead.

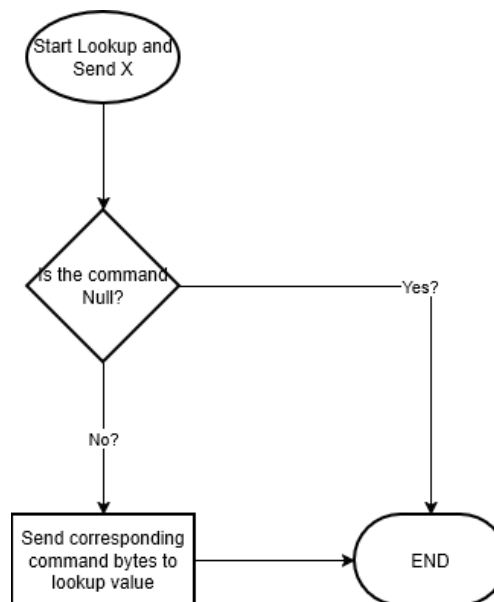


Figure 6.9: Lookup and Send Command Flowchart

There are two “lookup and send” processes in the code, however they accomplish the same task shown in Figure 6.9. They were split apart because the move commands have a different header than the button commands. By doing this, it helps to save space within their respective arrays. This process takes the stored command location and performs a “look up” inside of the array. This array is then indexed, sending the corresponding command bytes out of the TX pin of the PIC to the selected camera.

6.1.3: Camera Slave Circuit REV A

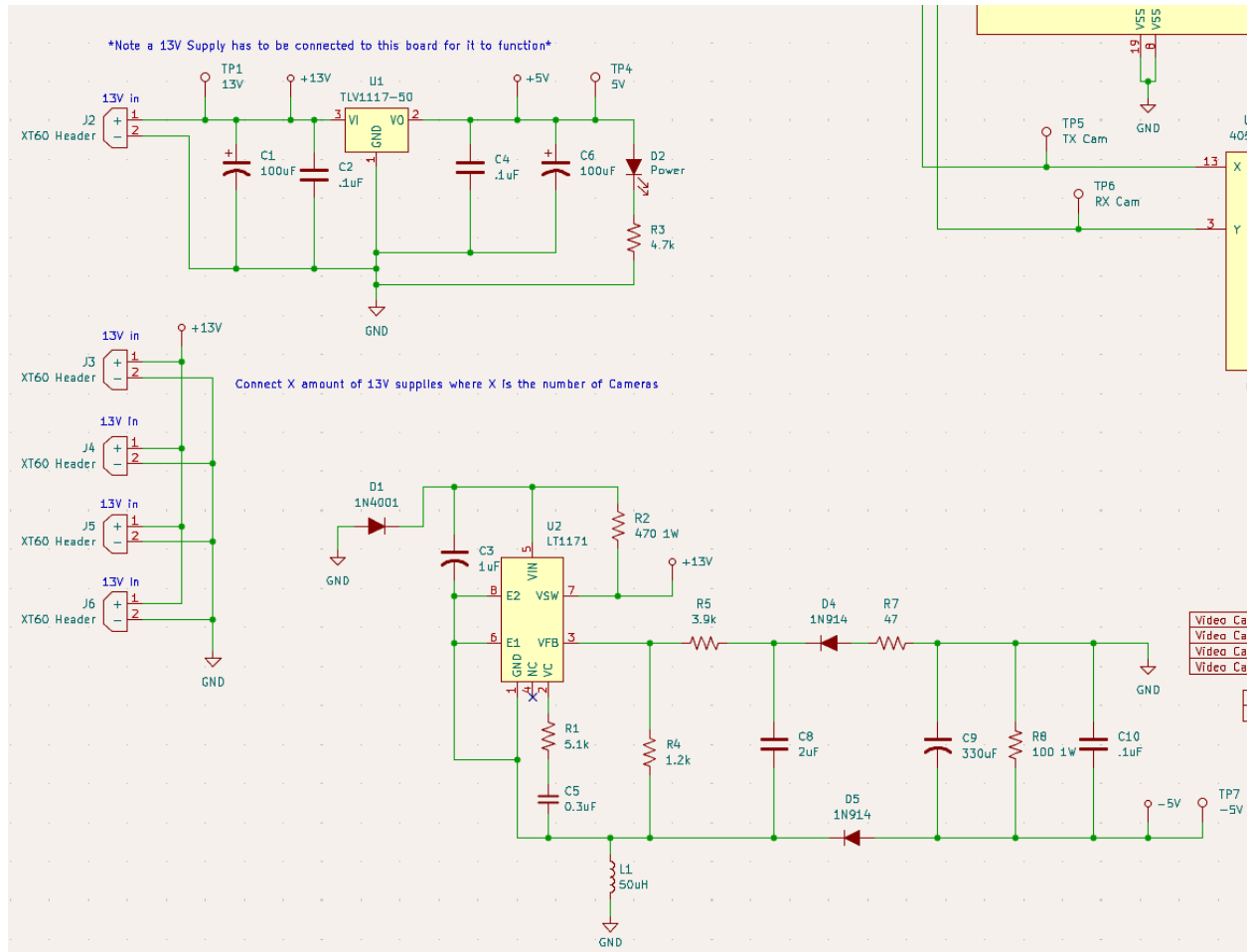


Figure 6.10: Power Schematic

The power inputs of the Camera slave board, shown above, are +13V buck regulators. These inputs are connected to the board and tied in parallel to maintain the +13V, but add the current from each one respectively. The +13V is then reduced to +5V via a linear regulator (U1) and -5V by a buck inverter (U2). The layout for the LT1117 was taken directly from the datasheet for that device with the recommended circuit design for 5V operation.

One problem with how the circuit's power is currently designed, is with the buck regulators being connected in parallel. With this configuration, they cannot output exactly the same voltage which would be required to tie them in parallel. For the current configuration, swamping resistors will need to be installed in series, between the positive output of the buck regulators and the board's positive inputs.

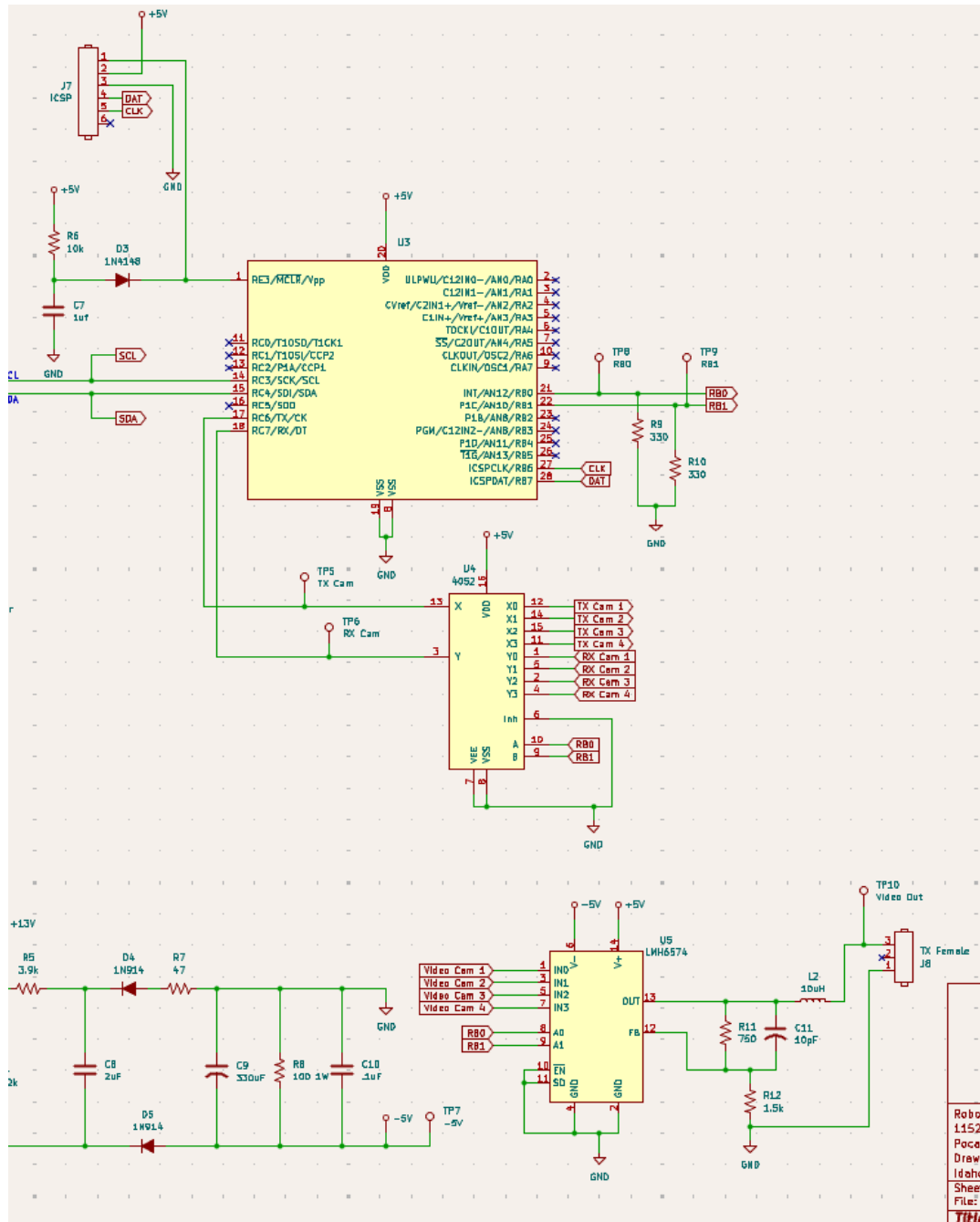


Figure 6.11: Digital Circuitry Schematic

The digital circuitry shown in Figure 6.11 consists of a PIC16F1788 (U3), a multiplexer (LMH6574, U5) and a dual demultiplexer (CD4052, U4). The PIC receives I2C on PORTC RC3 (SCL) and RC4 (SDA) which then gets translated into commands for the camera (see section 6.1.2 Camera Slave Program). The translated bytes for the camera then get outputted on RC6 (TX) into U4 to be demultiplexed. The demultiplexed signal then gets transferred to whichever camera is selected by PORTB RB0 and RB1. The TX signal from the camera (RX on the board) remains unused. The camera's video signal is brought back to the board and received by the video multiplexer (also selected by RB0 and RB1). The output of the multiplexer is an Op-Amp. The output gain of this Op-Amp should be 2, because the input impedance of the signal should be 75 ohms and then gets broadcasted on 75 ohms. The gain is set by feedback resistor R11 and negative input R12. There is then output filtering with C11 and L2, intended to filter out high frequency noise introduced by the system to keep the video signal clean.

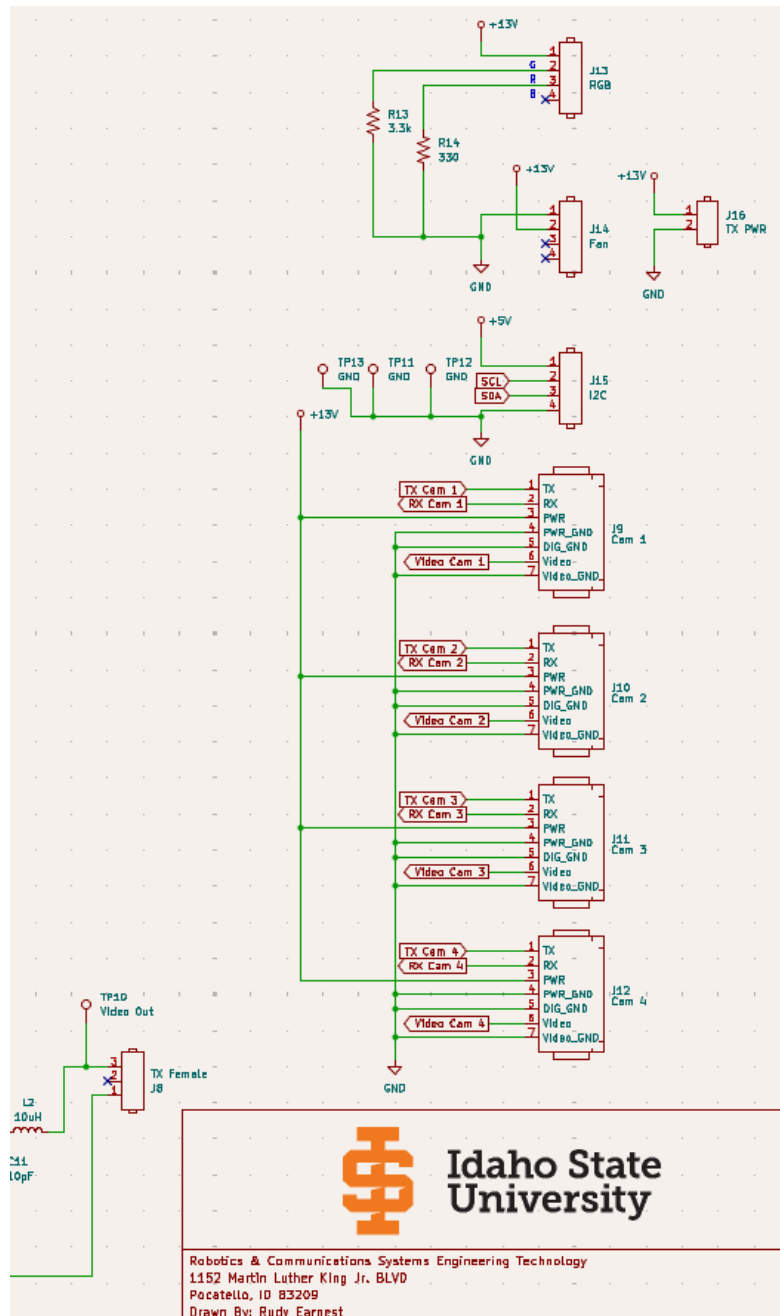


Figure 6.12: Outputs Schematic

The outputs of the board, shown in Figure 6.12, include four camera outputs, an I2C passthrough, headers for a 12V RGB fan to have a constant orange color, and headers for the transmitter with a power header and the

video header. The fan header output was intended to cool the 13V buck regulators.

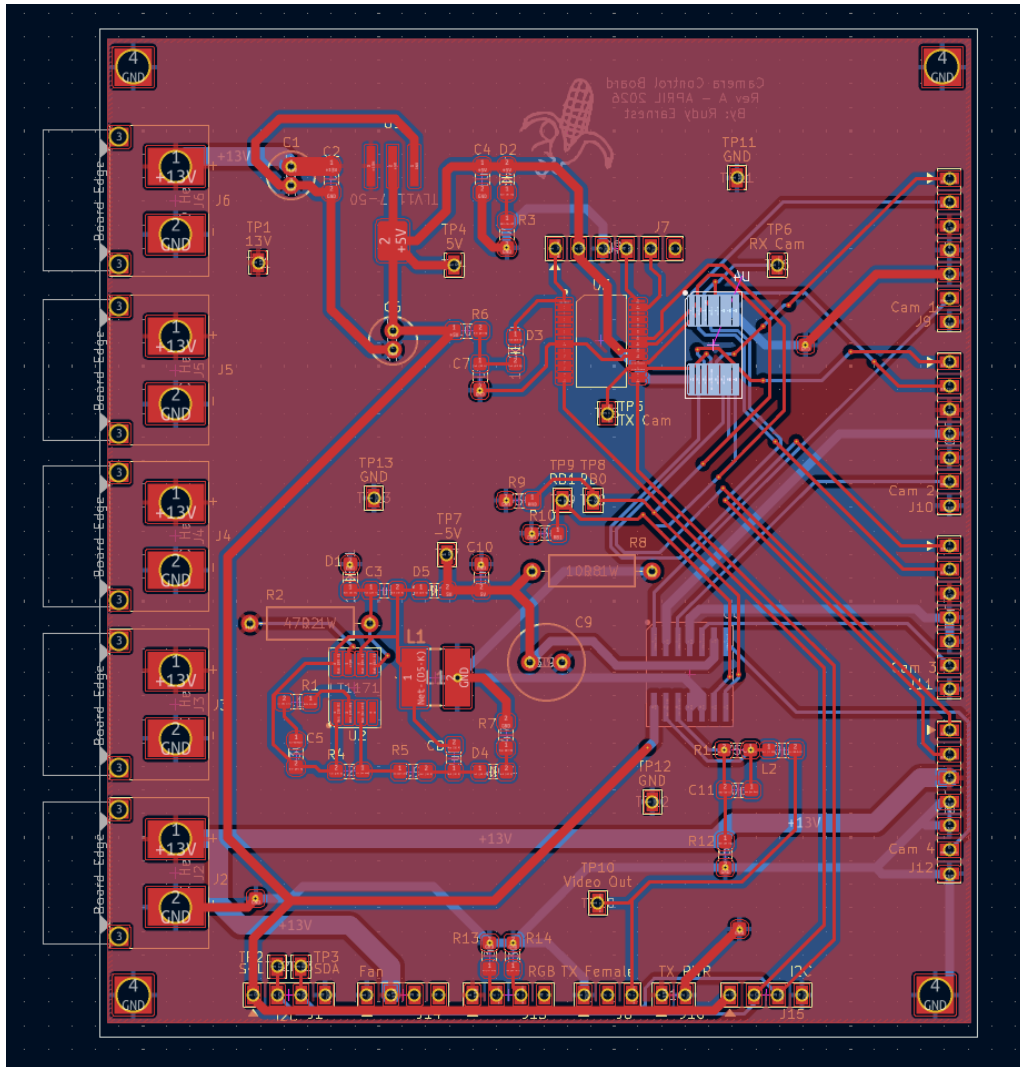


Figure 6.13: REV A PCB layout

The current layout of the PCB has Power inputs on the left and camera outputs on the right. The other outputs are along the bottom of the PCB including the I2C, TX outputs, and Fan outputs. Full PCB drawings will be included in the appendix.

6.1.4: Buck Converter

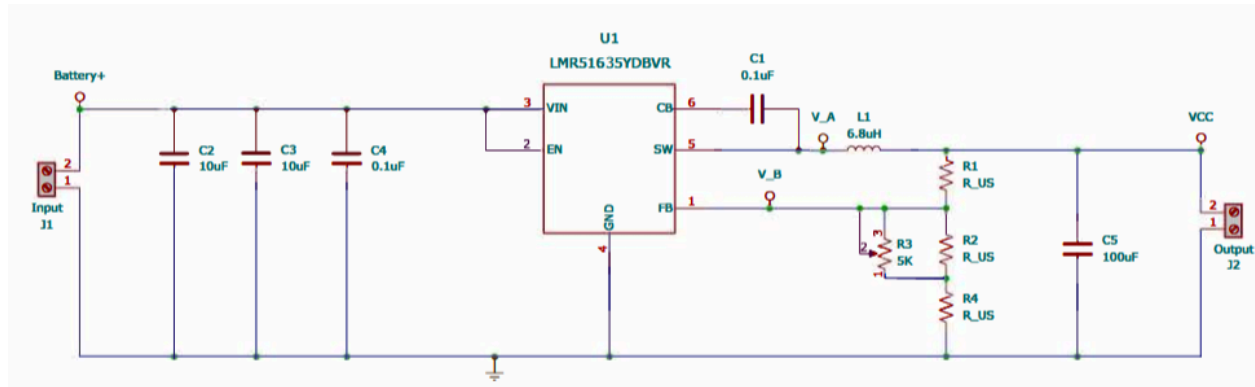


Figure 6.14: Buck Schematic

The buck circuit voltage is set by the resistors R1, R2 and R4 shown in Figure 6.14. The network of R1, R2, and R4 are set to have a +13V output. For the board's power input, as mentioned previously in section 6.1.3, there would need to be a swamp resistor inserted in series between the positive output wire of this buck regulator and the board.

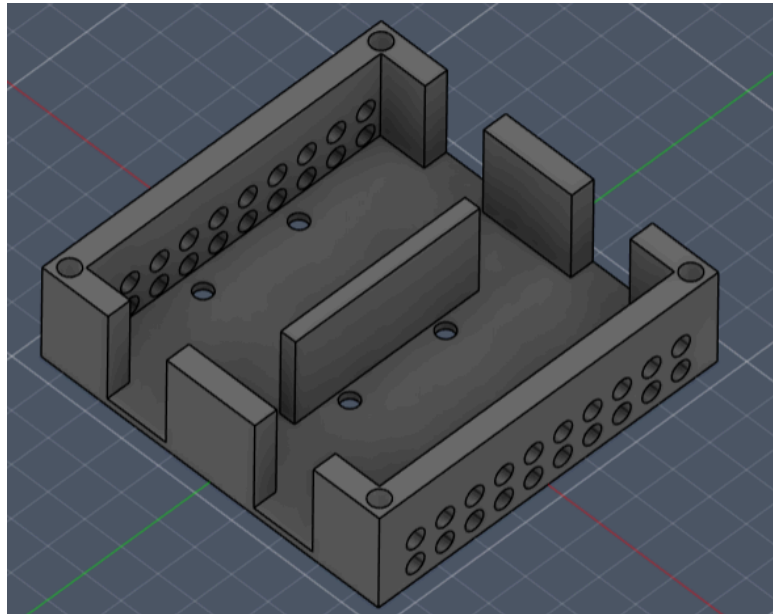


Figure 6.15: Fan Buck Case

Figure 6.15 shows the 3D model for a buck regulator case that uses a 90mm fan on top to cool the regulators. There can be two regulators inserted per case.

6.1.5: VB Remote Testing TX Mode Form



Figure 6.16 Universal Remote Testing TX Mode

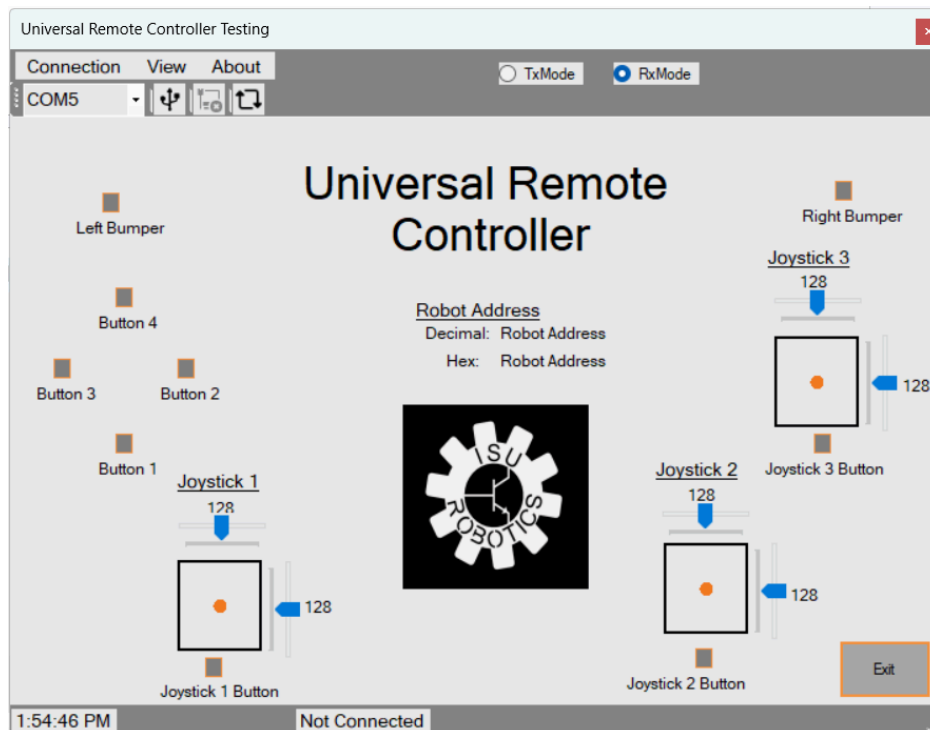


Figure 6.17: Universal Remote Testing RX mode

A form was created on top of the existing Universal Remote testing form for the purpose of testing without the physical hardware. The mode hides behind the TX mode radio button within the existing form. There are all of the buttons and “potentiometers” that would be on the physical hardware to simulate the presses and movements of the normal controller. With the use of this program the user would need to implement a MAX323 to change the RS-232 to TTL levels. Then place the Transmit out of the MAX232 to the RX on the remote master board and RX to TX. This then allows the master to send out the byte packets that should be expected over I2C allowing for the user to test a slave device.

6.2: LiDAR - Light Detection and Ranging

6.2.1: LiDAR Packet

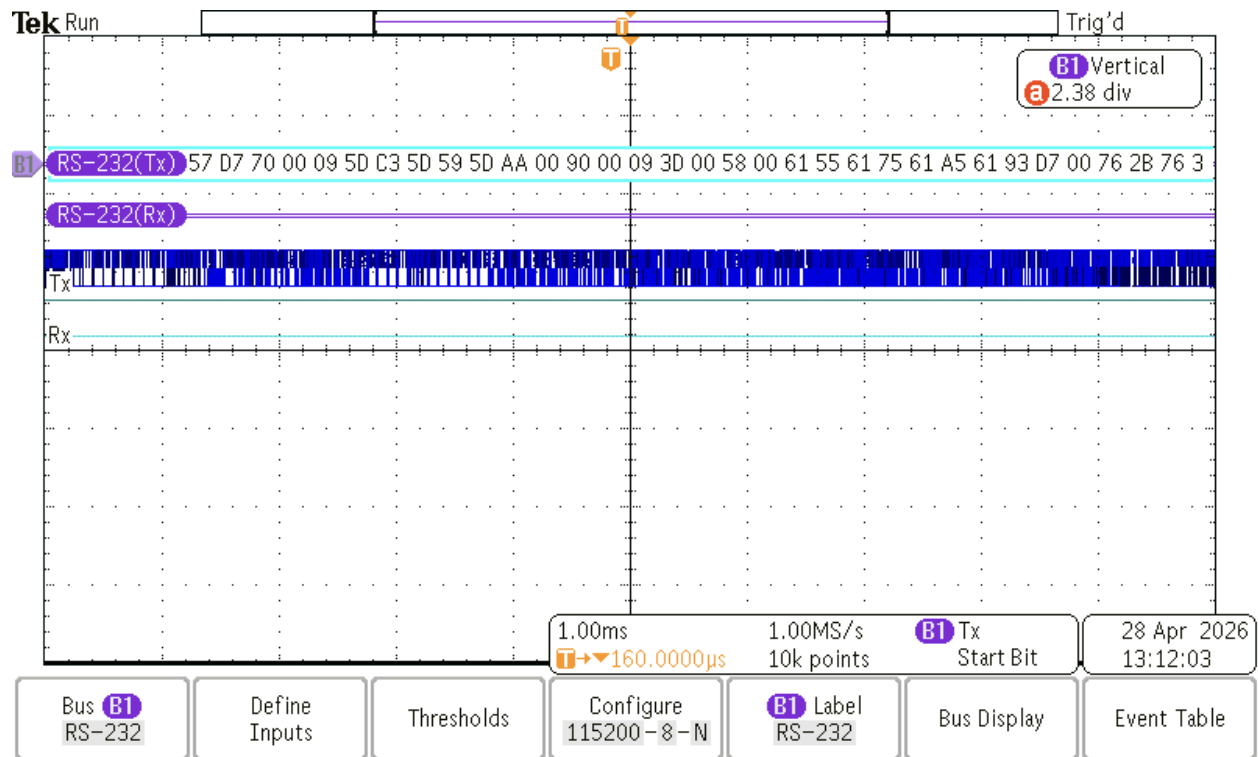


Figure 6.18: Raw LiDAR Data Waveform

The data collected from the LiDAR's UART cable was sent at a baud rate of 115,200 characters per second with 8-bit bytes. When analyzing the packet, there were no discernable start/stop bits, making it rather difficult to try and decipher where a packet started/stopped.

There was little to no documentation on how to read or process the data directly from the UART interface without using the manufacturer's software. Initially, the terminal output of the manufacturer's software was used to try and read some of the data from a "working state". When using the terminal output, most of the debug messages contained extra characters

and values denoting the time, date, and Mandarin characters specifying the native control/function of the software being performed. This resulted in much of the collected data from the instance being rendered useless.

6.2.2: Decoding the Data Packet

To remedy the lack of being able to see the full raw data sent from the LiDAR module, a basic Windows form (WinForm) was designed in C# to read the incoming data and write it to a file for analysis. When analyzing the stored file to attempt to gain an understanding of what protocol was being used or how to pick apart the data structure into sections, the only recurring recognizable pattern was the first two bytes “AA 55,” assumed to be the header/preamble for each packet, which was later verified to be a common value for LiDAR packet headers.

With the header now established as a known point, the C# program was modified to handshake the first two bytes of each received data packet and only accept/store the data if the conditional was fulfilled. Testing was done with the circuit in Figure 6.19.

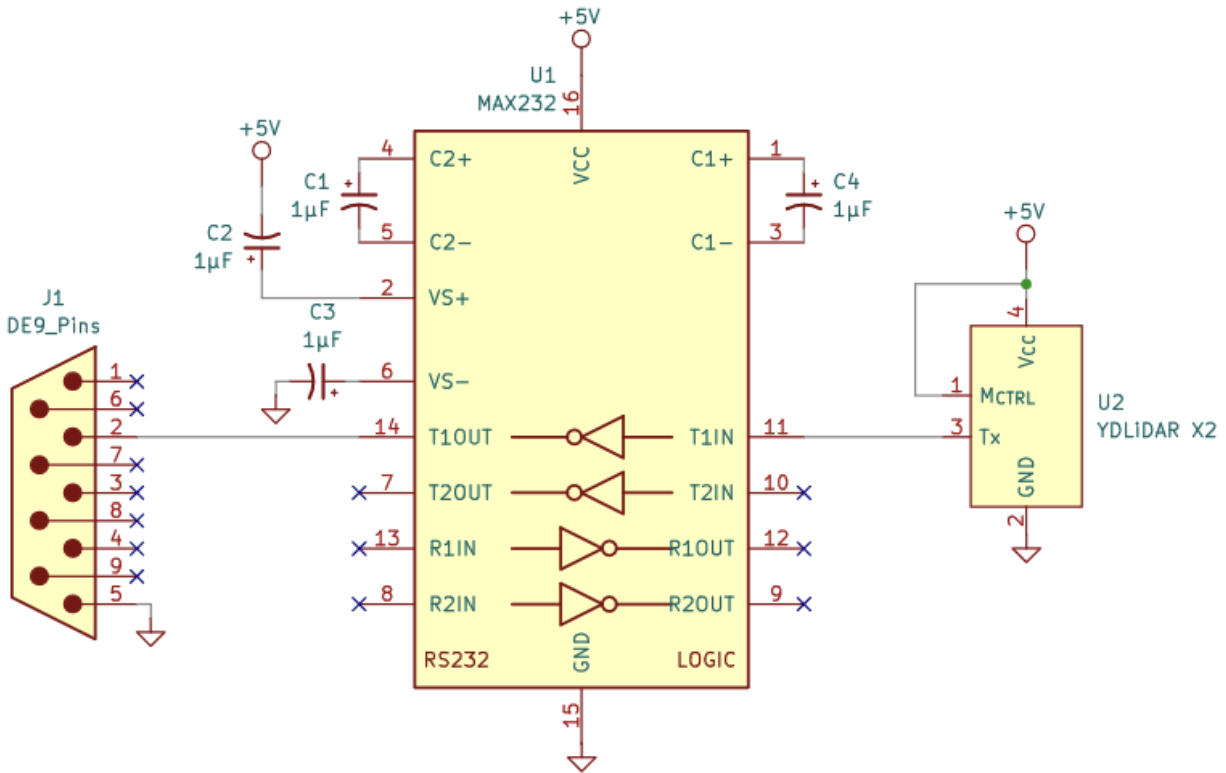


Figure 6.19: LiDAR TTL/RS-232 Interface Circuit

Acquisition (yyyymmdd. hhmm:ms)	Data
20260430.1040 :190	AA 55 00 28 4D 27 01 36 88 7C 28 38 F4 37 E1 37 30 37 99 37 6C 37 69 37 49 37 B2 2E 0A 32 D2 33 41 34 16 2F 4A 2D 59 2D 96 30 BA 31 72 35 72 30 E6 29 16 37 85 37 99 37 CD 37 7D 37 B4 37 00 00 00 00 A2 0E CE 09 A9 09 7D 09 65 09 49 09 2D 09 19 09 09 09 F9 08 E9 08 E5 08
20260430.1040 :190	AA 55 00 28 59 36 0F 45 02 4C E5 08 ED 08 04 09 3A 0C 4D 0C 5D 0C 75 0C A1 0C CD 0C 09 0D 49 0D 95 0D BE 02 B9 02 BD 02 C5 02 C9 02 CD 02 D5 02 DD 02 E1 02 E9 02 F1 02 F9 02 FD 02 0D 03 00 00 00 00 00 00 CE 02 D9 02 5A 05 00 00 00 00 00 00 00 00 00 00 00 00 00 DA 42

20260430.1040 :190	AA 55 00 27 93 9F CB AD 63 78 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 7A 07 00 00 00 00 00 00 00 00 00 00 BE 04 DC 04 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 32 06 30 06 25 06 3C 06 A6 06 00 00 00 00 00 00 00 00 00 00 00 00 00 00 B6 03 0E 0E DC 0D A0 0D DA 12 2A 26
20260430.1040 :190	AA 55 01 01 2D AE 2D AE AB 54 00 00

Table 6.3: Formatted/Indexed LiDAR Data

With the LiDAR's UART data now being able to be easily read from a file, the C# program was modified to attempt and interpret the data packet into data easier for the user to understand. By interpreting the data inside of a C# program, this would hypothetically make the process of translating from a WinForm to the PIC.

Unfortunately, the logic for converting the data packet into readable information is NOT correct and displays very incorrect information. This is believed to be caused by an incorrect time-based sequence rather than an event-based sequence for the UART. Fortunately, the WinForm does process the data, albeit incorrectly, and displays it with minimal issues during operation. The GUI output can be seen in Figure 6.20 below.

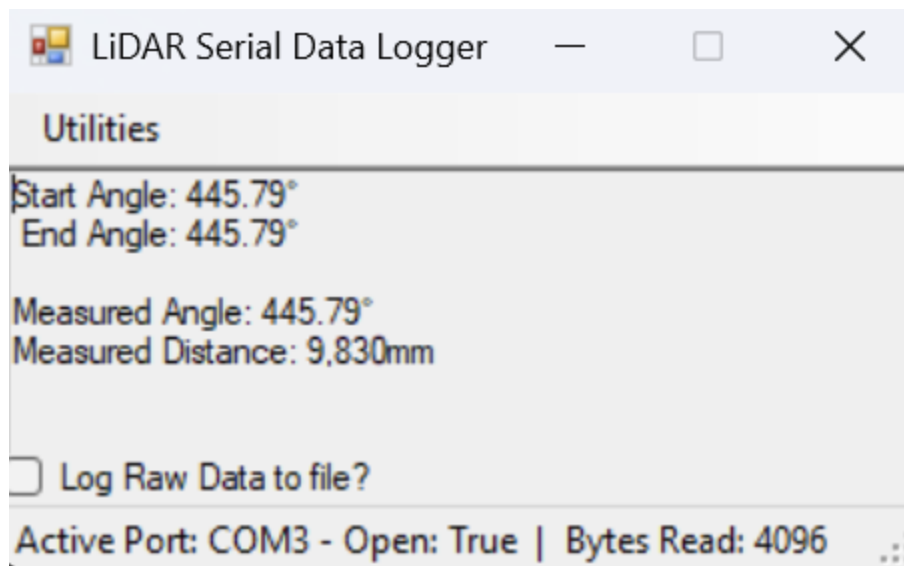


Figure 6.20: C# GUI - Data Translation Attempt Output

7: Troubleshooting Procedure

7.1: Camera Slave Troubleshooting

7.1.1: Power Verification

7.1.1.1: Buck/Regulator Output Input

- Verify the buck converter/regulator is outputting approximately +13.8V DC.

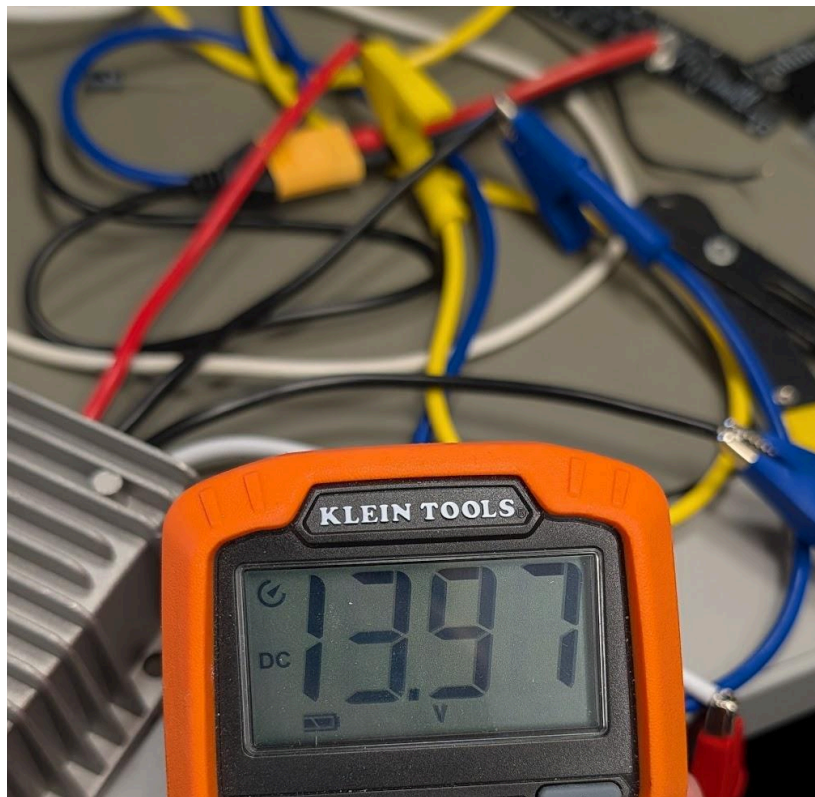


Figure 7.1: Expected Buck/Regulator Voltage

- Using TP1 (ground references at TP11, TP12 and TP13) verify that the board is receiving approximately +13.8V DC.

- If 13V is not present at TP1 check XT60 connections and reflow any cold or incomplete solder joints present at the XT60 header at the board.

7.1.1.2: +5V Linear Regulator Output (TP4)

- Using TP4 (ground references are TP11, TP12 and TP13) verify that the linear regulator is outputting +5V DC

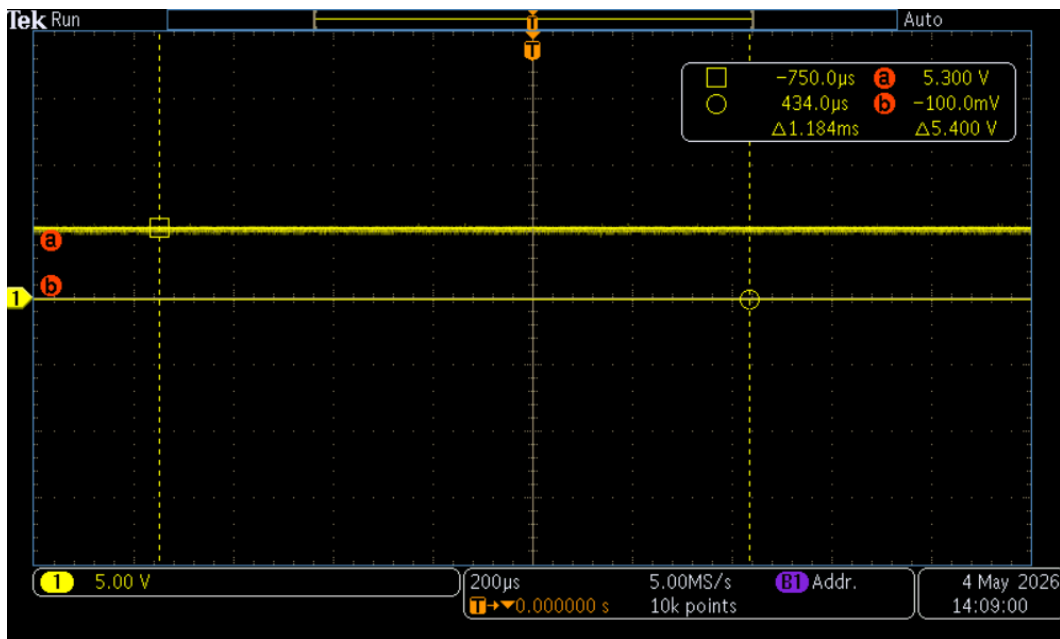


Figure 7.2: TP4 Expected Voltage

- If +5V is not present at TP4 inspect solder connections of components connected to U1.
- If the issue persists, replace the regulator on the board.
- If +13V present at TP4 replace all components connected to +5V **after replacing the regulator with the output showing it is +5V.**

7.1.1.3: -5V Buck Inverter (TP7)

- Using TP7 (ground references are TP11, TP12 and TP13) verify that -5V is present.

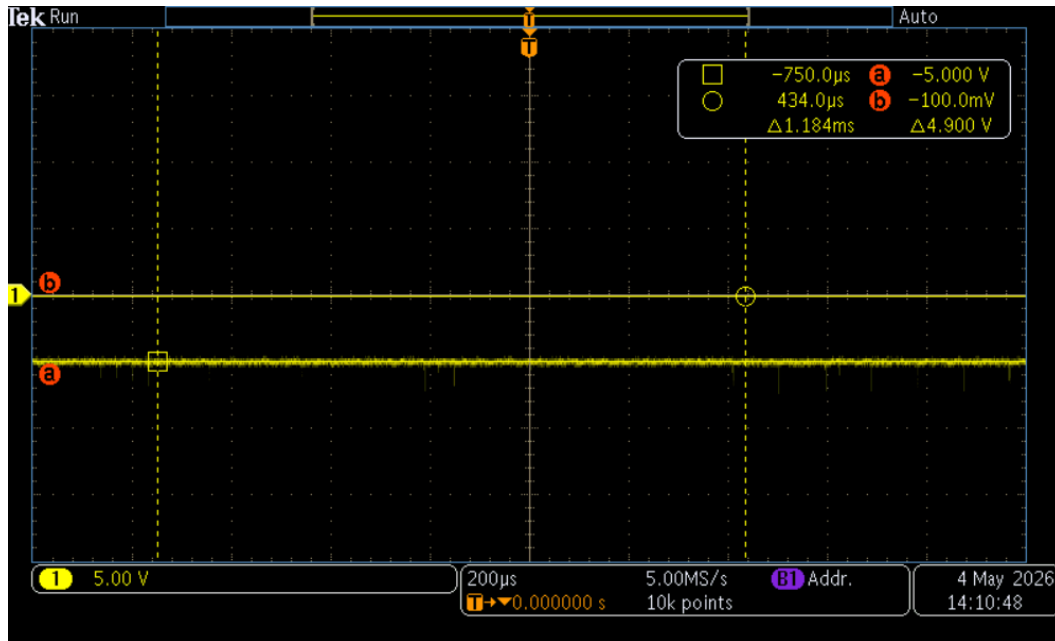


Figure 7.3: Expected TP7 Voltage

- If -5V is not present inspect solder connections connected to U2.
- Ensure R2 and R8 are the correct wattage as specified on the schematic.
- If the issue persists, replace U2 on the board.
- If a voltage of +-13V was found at TP7 replace U5 after confirming a -5V at TP7.

7.1.2: Digital/Control Signal Verification

7.1.2.1: I2C Bus from Master/RX Controller (TP2 & TP3)

- TP2 should have the SCL signal from the Master, and TP3 should have the SDA signal from the master.

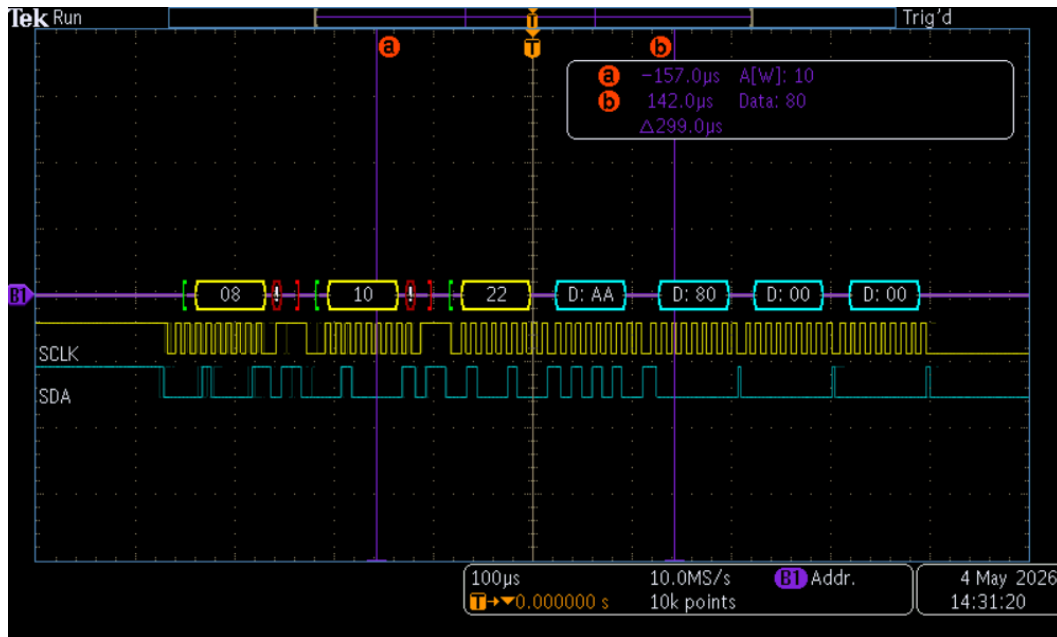


Figure 7.4: Expected TP2 and TP3 Waveform

- Verify that the respective SCL and SDA signals are present at TP2 and TP3.
- If the signals are not present verify that the I2C header connection is firmly seated on the board. Verify that the Master/RX controller is actively outputting the correct I2C signal.
- *If there is no controller available for testing, the VB controller simulator can be used with an RS-232 to USB connected to a MAX232 IC, with the TX and RX pins connected to the RX and TX of the Master board.*

7.1.2.2: Slave UART Output (TP5 & TP6)

- TP5 is the TX of the UART i.e. commands to the camera, while TP6 is the RX which will have the TX from the camera on it which is currently unused and reserved for future system updates.

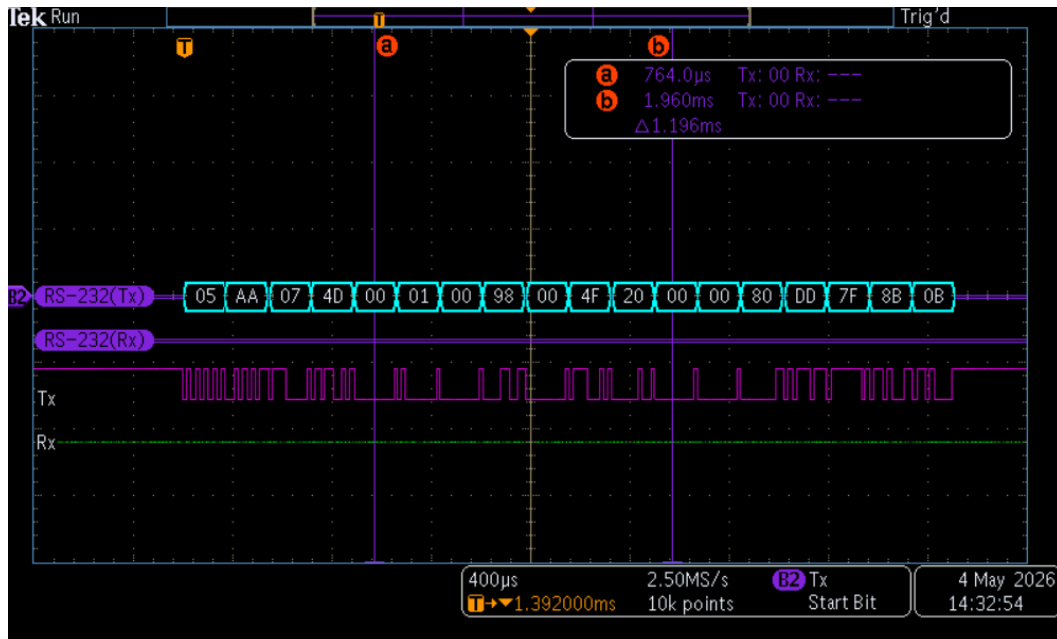


Figure 7.5: Expected TP5 and TP6 Waveform

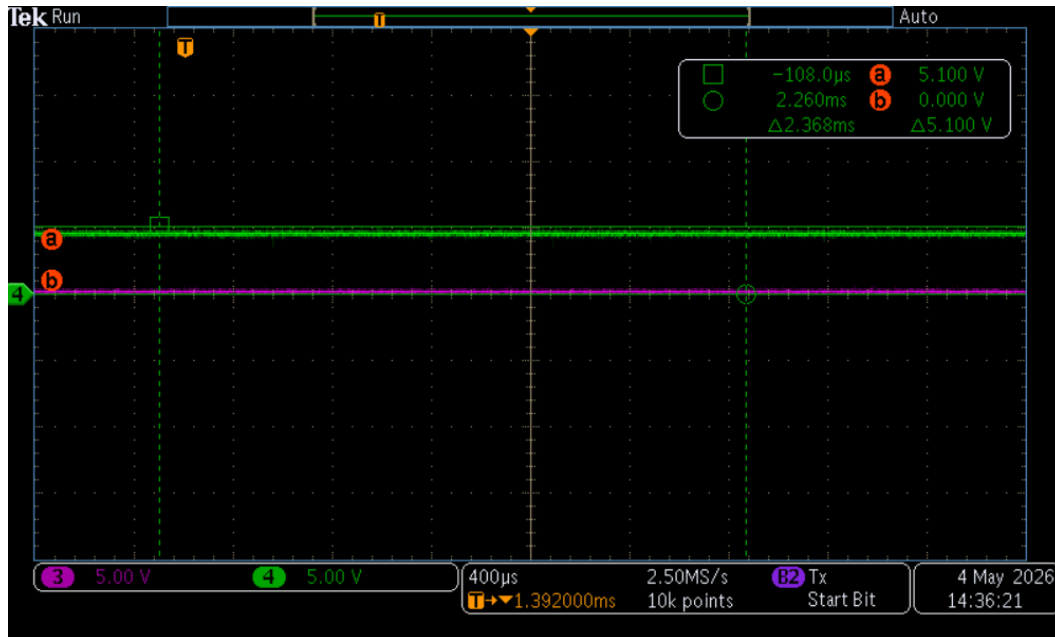
- If the signal is not present at TP5 when a button is pressed or joystick 3 is moved, ensure that the board is loaded with known working code. If the signal is still absent, inspect the PIC16F1788 for cold solder joints and damage.
- If the problem persists, replace the PIC16F1788 U3 on the board.

7.1.2.3: Camera Address Bits (TP8 & TP9)

Camera Selected	TP9 (MSB)	TP8 (LSB)
Camera 1	0	0
Camera 2	0	1
Camera 3	1	0
Camera 4	1	1

Table 7.1: Camera Address Bits

- The bits of TP9 and TP8 should initialize to 00 (Camera 1) and increment each time the X button is pressed. Note the X button has to be pressed and then released and then pressed again to increment TP8 and TP9.

**Figure 7.6: TP8 and TP9 Camera 2 Selected**

- Verify that the camera being troubleshooted is the camera selected by TP9 and TP8.

- Confirm that the proper sequence occurs.
 - If the increment fails or is out of order, ensure working code has been loaded onto the slave.

7.1.2.4: Camera output header

- Once TP8 and TP9 have been verified, check Pin 1 of the respective camera header to see that TP5's signal is on Pin1.
- If the signal has been verified to be on Pin 1 and the camera is still not operating with the correct commands, replace the camera with a known working unit.

7.1.3: Video Signal Troubleshooting

7.1.3.1: Confirm Camera Address (TP8 & TP9)

- Refer to section 7.1.2.3 for correct camera addressing.

7.1.3.2: Board Video Signal (TP10)

- Verify that an NTSC composite video signal is present on TP10

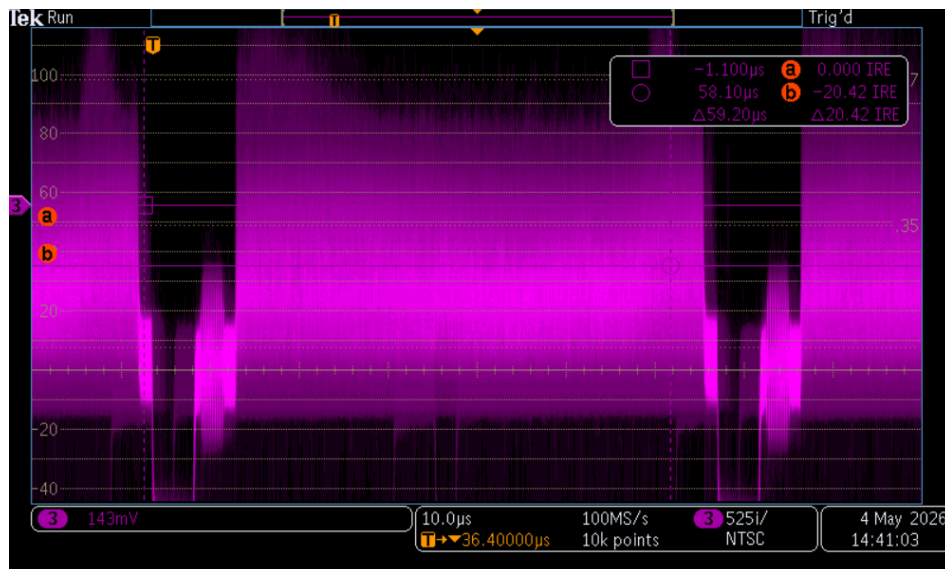


Figure 7.7: Expected TP10 Waveform

- If the signal is not present, power the camera independently from the board and verify an NTSC composite signal is being transmitted.
- If the camera is not outputting a video signal, replace it with a known working unit. Otherwise, inspect the camera's header for proper connection and solder.
- If no signal is present on TP10, inspect U5's circuit to verify it is soldered properly.

7.1.3.3: RF Transmission Verification

- If no signal is being received, deploy a spectrum analyzer to determine if the transmitter is actively broadcasting.

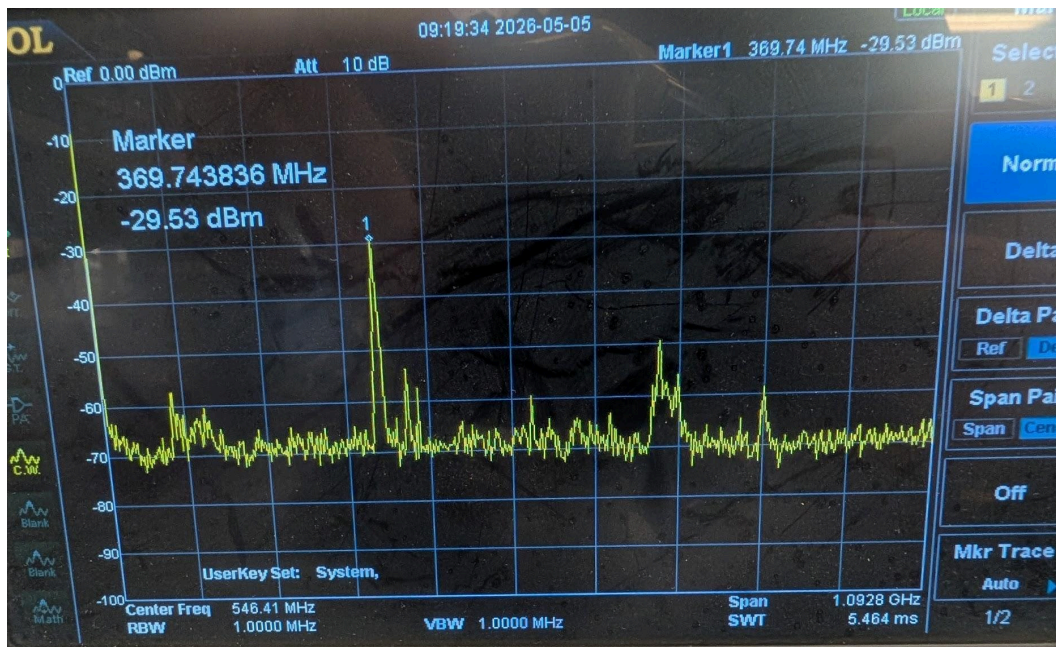


Figure 7.8: Example Spectrum Analysis

- To identify if the signal seen is the transmitter or not, turn off the circuit or depower the transmitter. If the signal goes away that was the transmitted signal and if it wasn't, look for another signal until all peaks have been tested.
- If the transmitter has not been found to be broadcasting, replace the transmitter with a known working unit, if possible.
- Once the transmitter has been verified, press the search button on the receiver. It should automatically detect and display the video signal.
- If the receiver is unable to detect the video signal, replace the receiver with a known working unit, if possible.

8: Cost Analysis

The overall cost of this project was \$97,224.51. The camera board and components cost \$54.52, while the cameras' total cost was \$82,070.00, representing the bulk of the project cost. The universal remote boards, including the two boards and their components, cost a total of \$255. The LiDAR system cost \$69.99 for the LiDAR itself, currently being the sole contributing factor for this price, seen in Table 10.1. The individual times for each member were 232.5hrs (Table 10.2) and 63hrs (Table 10.3) respectively, billed at 50\$ an hour, accounting for \$14,775.00 of labor expenses.

The time utilized for this project was mainly spent developing the systems for the camera, for either control logic for the camera or designing circuits and IC selection to be utilized on the PCB, also designed this semester. The other time was spent on trying to understand the specific 360 LiDAR module for this project. Most of the time spent on the device was searching for device documentation and how to interpret the received packet or find an explanation of the packet structure. The time for the camera could have been better spent by designing the systems quickly and efficiently rather than spending days at a time on the specifics of design, especially pertaining to the 3D model for the regulator case.

9: Conclusion & Recommendations

The goals of implementing multiple camera control, microcontroller control, refining camera interface, I2C protocol for the camera (with the universal remote), were successfully completed. Additional goals, like implementing other sensors and making this project more “open-sourced,” allowing for potentially easier integration with other projects utilizing the universal controller system.

The list of failed project objectives include integrating a functioning incorporation of the LiDAR system, onboard recognition/object detection from the native cameras’ video feed, and developing a TX/RX protocol for the camera and other sensor data.

The main roadblocks that prevented the team from completing all goals were mainly due to time constraints and the inability to design systems quickly enough for full utilization. Specifically, with the camera, the TX/RX and object detection were taken into consideration but due to the lack of available time, this feature was unfortunately not implemented this semester.

Improvements that could be made to preexisting components are the PCB boards with a REV B. This improvement request stems from the PCB’s multiple parallel outputs used. The main problem as mentioned in section 6.1.3 is that swamping resistors would have to be placed in series with the positive input of the buck regulator to the power input of the board. During test trials with this implemented correction, it was discovered that even

with two buck regulators in parallel, the output lacked a stable voltage level and sufficient current output for a single thermal camera. To remedy this critical flaw, a 13V 10A regulator was ordered, rendering the current board design obsolete.

Another recommended improvement would be to find and utilize a camera stabilization command if such command exists. When a camera is connected to the provided camera control software from Hood Tech, the camera either minimally shifts or not at all, meaning that there could be a camera stabilization command. This would need to be “sniffed out” (read from the TX from the Hood Tech software) when the camera is connected and at an idle position that is not home. This may prove difficult as the program is always sending data and the command may not appear in the same format (have the same first 9 bytes) as the rest of the project program’s commands.

Considering that a board design will be necessary in the future, possible expansions could be adding commands for a potential variety of PTZ cameras, as well as selecting new components. Selecting multiplexers and demultiplexers that have more outputs/inputs would increase the possible number of cameras allotted per board. Another addition would be the incorporation of an NVIDIA Jetson for video processing and object recognition, specifically for the Bloodhound project. This would significantly aid operations within search and rescue missions that the Bloodhound is designed for. With the integrated recognition software for

human-shaped objects and heat signatures. As mentioned in section 7.1.2.2, the RX on the board is currently unutilized. The board is capable of receiving the transmitted camera signal, but currently is unable to be decoded for dictating camera positioning, transitioning from a single-sided to a dual-sided positioning system. This is something that may be useful in future projects requiring motor position data from the camera to possibly incorporate automatic control within an object tracking system.

In the future this project can be utilized by any project already utilizing the universal controller. This project can then be used granting the option user access to control of the Hood Tech cameras, even if that is in a diminished capacity relative to this project. The code itself allows the user to reconfigure the “mapping” or placement of camera commands on the controller. Unwanted or unused buttons can be disabled by placing a null value (0xFF) for the button press rather than the lookup value. With the possibility of implementing object recognition with the NVIDIA Jetson device added to a project, this significantly increases the number of possible uses for this project with its involvement controlling PTZ cameras. This could even be expanded with further developed functionality of a process to allow the Jetson to control the position of the camera and move it in order to track an object.

Recommendations for the LiDAR module would be to not purchase inexpensive LiDAR modules from a non-reputable merchant/source, because of the significant lack of documentation available. If one were to

continue with the work done on this project, the recommendations would shift to focus on the bugs and issues present with the C# UART program; Correct the known or discovered bugs/issues and perform a scan operation with controlled angle sector and depth/distance. As mentioned in its own section, the program relies heavily on a timed-based sequence rather than an event-based sequence. Some progress was made to transition to this better fix, but still requires correction. Theoretically, the proposed event-based sequence could be greatly simplified if the packet structure were documented as well as determining if there were any “stop” bits present at the end of the packet.

In short, this project, specifically PTZ camera control, has the capacity to be utilized in numerous projects with many different use cases because of the mostly open-ended integration allowance.

10: Appendix

10.1: 3D models

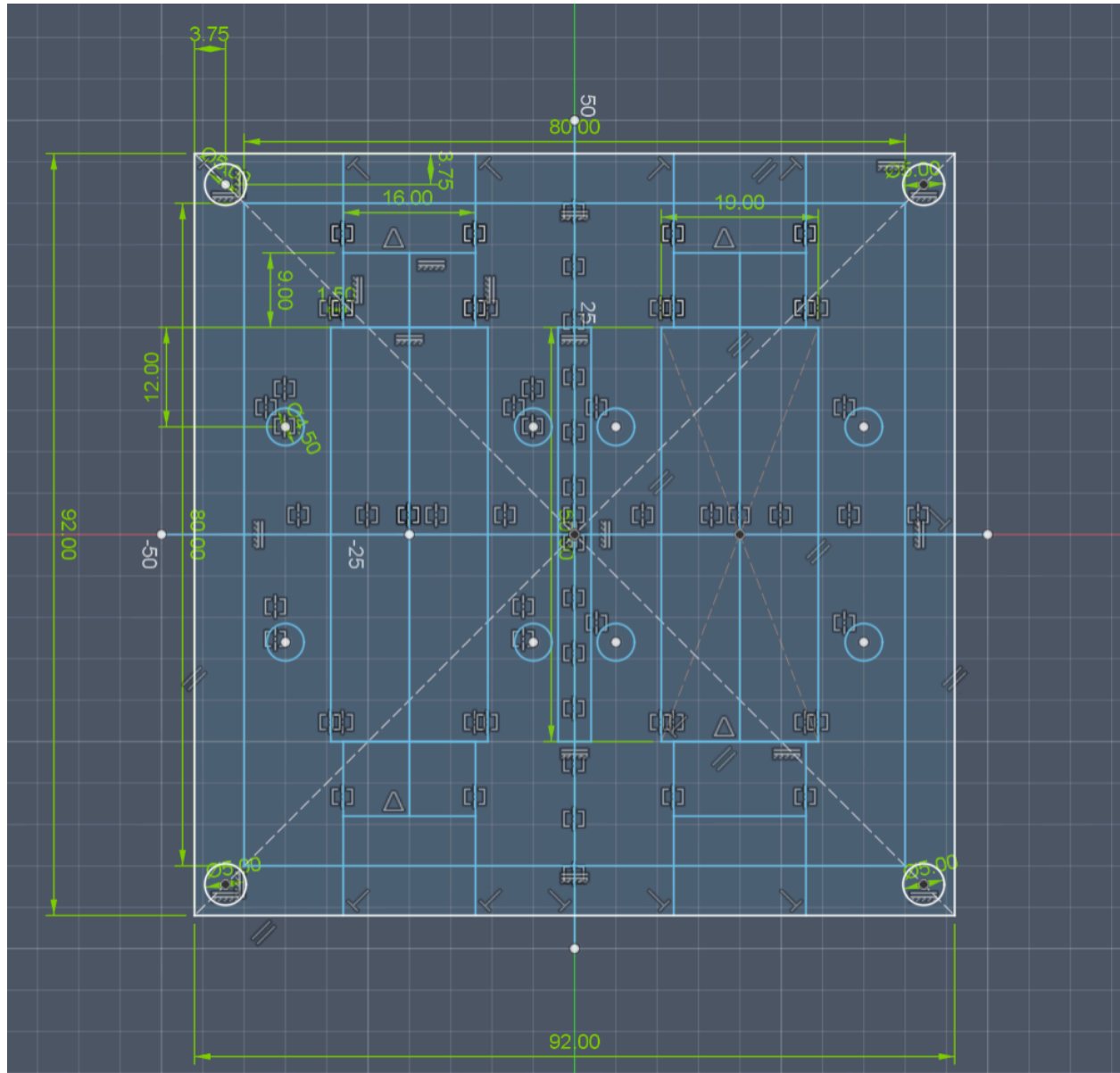


Figure 10.1: 90mm Fan Buck Case Sketch

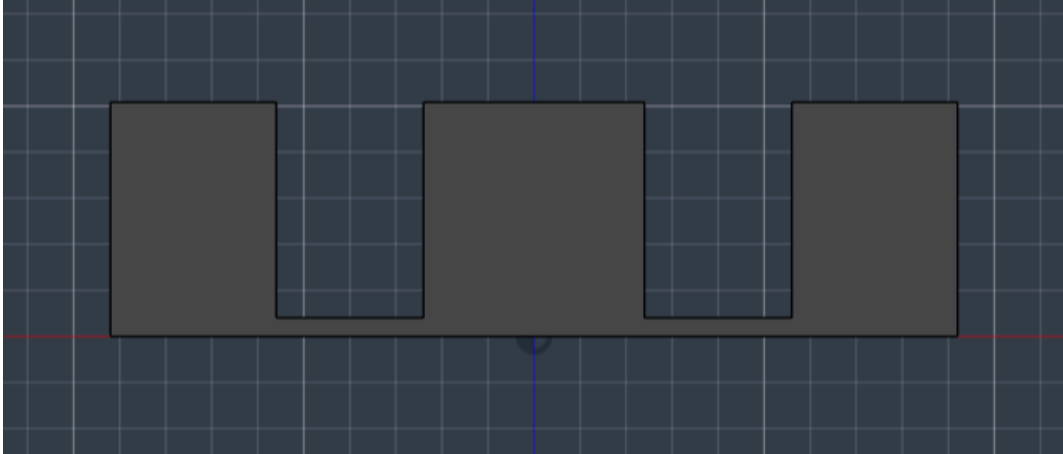


Figure 10.2: Front View

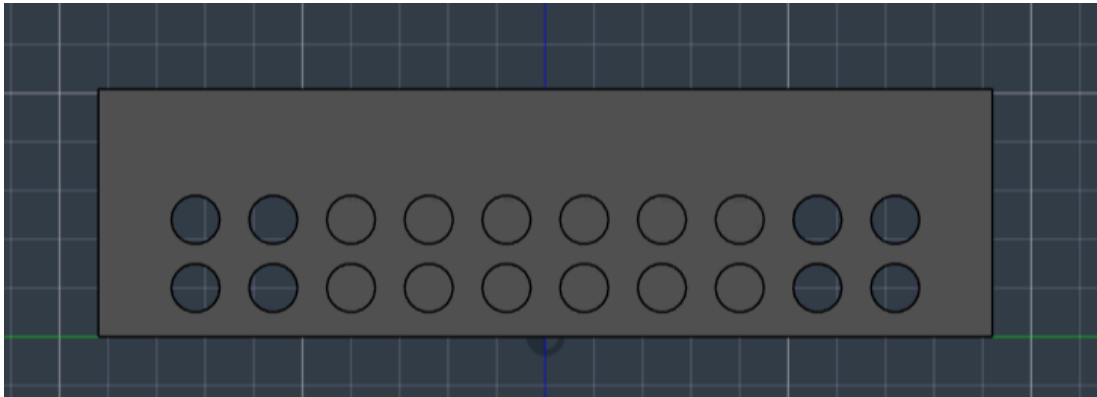


Figure 10.3: Side View

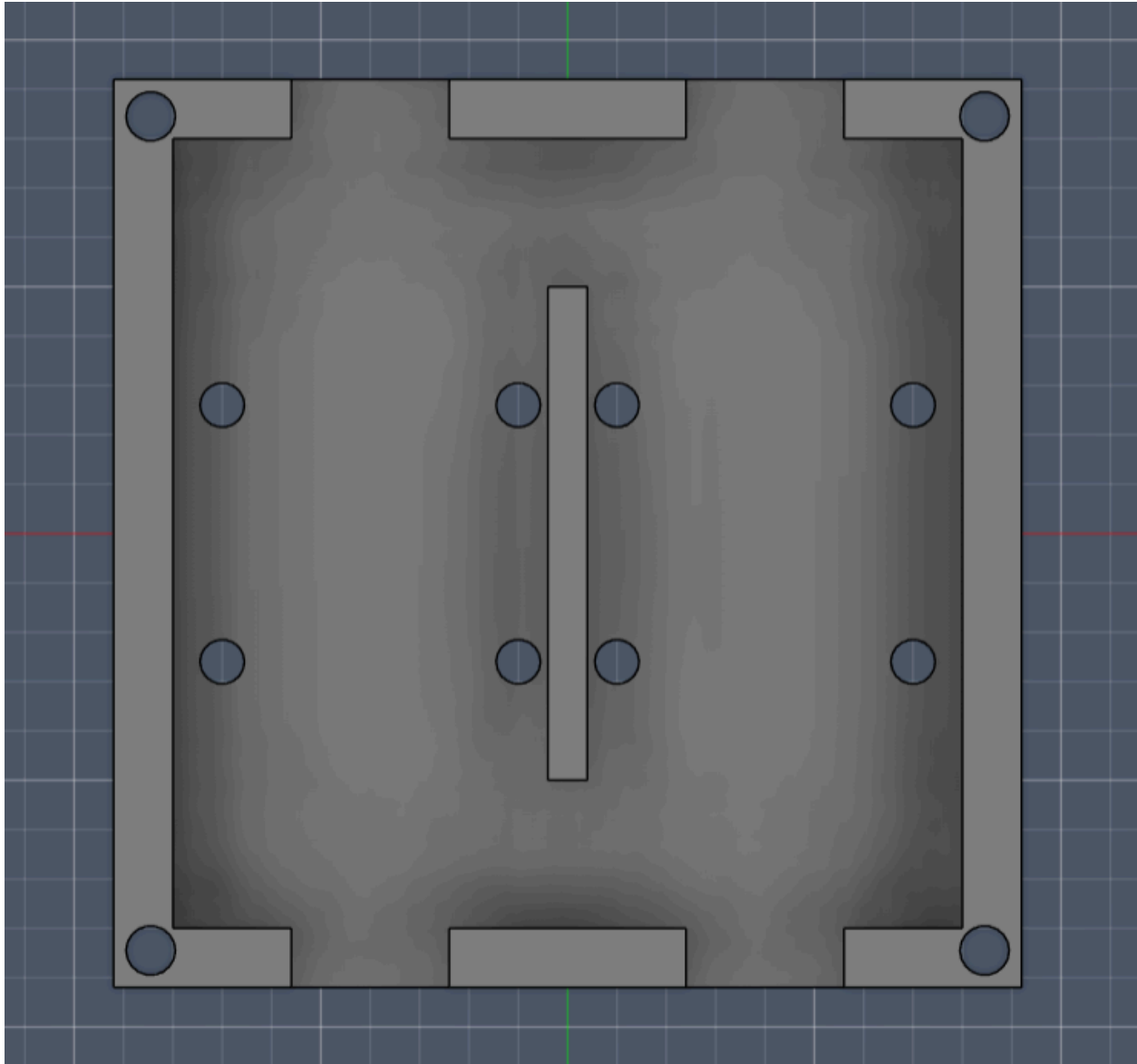


Figure 10.4: Top View

10.2: BOM

Reference	Value	Footprint	Qty	Part #	Unit \$ @ 100	Cost Per Board	
C1,C6	100uF	Capacitor_THT:C_Ra dial_D5.0mm_H11.0 mm_P2.00mm	2	107CKE0 25MEM	\$0.13	\$0.26	
C2,C4,C10	.1uF	1FootPrintLibrary:080 5	3	VJ0805Y1 04KBAAT	\$0.17	\$0.50	
C3, C7	1uF	1FootPrintLibrary:080 5	2	08053G1 05MAT2A	\$0.07	\$0.14	
C5	0.3uF	1FootPrintLibrary:080 5	1	C0805F3 34K5RAC TU	\$0.13	\$0.13	
C8	2uF	1FootPrintLibrary:080 5	1	C0805C2 25K4RAC 3123	\$0.14	\$0.14	
C9	330uF	Capacitor_THT:C_Ra dial_D8.0mm_H11.5 mm_P3.50mm	1	EEU-EB1 E331	\$0.27	\$0.27	
C11	10pF	1FootPrintLibrary:080 5	1	0805N100 G250CT	\$0.03	\$0.03	
D1	1N4001	1FootPrintLibrary:DIO DE_0805	1	PMEG100 50ELP-Q X	\$0.27	\$0.27	
D2	Power	1FootPrintLibrary:DIO DE_0805	1	IN-S85CS 5B	\$0.11	\$0.11	
D3	1N4148	1FootPrintLibrary:DIO DE_0805	1	1N4448W S-HE3_A- 08	\$0.28	\$0.28	
D4,D5	1N914	1FootPrintLibrary:DIO DE_0805	2	1N4448W S-HE3_A- 08	\$0.28	\$0.56	
J1,J15	I2C	1FootPrintLibrary:He ader_1_x_4	2	2.54 pitch header		\$0.00	
J2,J3,J4,J5, J6	XT60 Header	1FootPrintLibrary:XT- 60 Power Male (Load Side)	5	NA	\$9.96	\$9.96	
J7	ICSP	1FootPrintLibrary:He ader_1_x_6	1	2.54 pitch header	\$5.50	\$5.50	

Bloodhound Auxiliary Sensors 63

J8	TX Female	1FootPrintLibrary:Header_1_x_3	1	2.54 pitch header		\$0.00	
J9	Cam 1	1FootPrintLibrary:Header_1_x_7	1	7pin JST Male	\$9.00	\$9.00	
J10	Cam 2	1FootPrintLibrary:Header_1_x_7	1	7pin JST Male		\$0.00	
J11	Cam 3	1FootPrintLibrary:Header_1_x_7	1	7pin JST Male		\$0.00	
J12	Cam 4	1FootPrintLibrary:Header_1_x_7	1	7pin JST Male		\$0.00	
J13	RGB	1FootPrintLibrary:Header_1_x_4	1	2.54 pitch header		\$0.00	
J14	Fan	1FootPrintLibrary:Header_1_x_4	1	2.54 pitch header		\$0.00	
J16	TX PWR	1FootPrintLibrary:Header_1_x_2	1	2.54 pitch header		\$0.00	
L1	50uH	1FootPrintLibrary:78404064680	1	78404064680	\$1.05	\$1.05	
L2	10uH	1FootPrintLibrary:0805	1	HCM1A0805V2-100-R	\$0.97	\$0.97	
R1	5.1k	1FootPrintLibrary:0805	1	CRCW08055K10JNTA	\$0.05	\$0.05	
R2	470 1W	Resistor_THT:R_Axial_DIN0309_L9.0mm_D3.2mm_P12.70mm_Horizontal	1	OM4715E-R58	\$0.12	\$0.12	
R3	4.7k	1FootPrintLibrary:0805	1	AA0805FR-074K7L	\$0.02	\$0.02	
R4	1.2k	1FootPrintLibrary:0805	1	SR0805JR-7W1K2L	\$0.07	\$0.07	
R5	3.9k	1FootPrintLibrary:0805	1	CR0805-FX-3901ELF	\$0.01	\$0.01	
R6	10k	1FootPrintLibrary:0805	1	MCU0805MD1002BP500	\$0.23	\$0.23	
R7	47	1FootPrintLibrary:0805	1	RC0805DR-0747RL	\$0.03	\$0.03	

Bloodhound Auxiliary Sensors 64

R8	100 1W	Resistor_THT:R_Axia I_DIN0309_L9.0mm_ D3.2mm_P12.70mm_ Horizontal	1	CMF6010 0R00FHE B	\$0.33	\$0.33	
R9,R10,R14	330	1FootPrintLibrary:080 5	3	CR0805-J W-331EL F	\$0.01	\$0.04	
R11	750	1FootPrintLibrary:080 5	1	CRCW08 05750RF HEAP	\$0.08	\$0.08	
R12	1.5k	1FootPrintLibrary:080 5	1	AF0805F R-071K5L	\$0.03	\$0.03	
R13	3.3k	1FootPrintLibrary:080 5	1	CR0805-F X-3301EL F	\$0.02	\$0.02	
TP1	13V	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP2	SCL	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP3	SDA	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP4	5V	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP5	TX Cam	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP6	RX Cam	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP7	-5V	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP8	RB0	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP9	RB1	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP10	Video Out	1FootPrintLibrary:He ader_1_x_1	1	2.54 pitch header		\$0.00	
TP11,TP12, TP13	GND	1FootPrintLibrary:He ader_1_x_1	3	2.54 pitch header		\$0.00	
U1	TLV1117- 50	1FootPrintLibrary:TLV 1117-50	1	TLV1117- 50IDCY	\$0.51	\$0.51	
U2	LT1171	1FootPrintLibrary:LT1 172	1	LT1172CS 8#TRPBF	\$6.69	\$6.69	

U3	PIC16F1788	1FootPrintLibrary:PIC16F883	1	PIC16F1788-E/SS	\$2.35	\$2.35	
U4	CD4052	1FootPrintLibrary:CD4052BPWR	1	CD4052BPWR	\$0.30	\$0.30	
U5	LMH6574	1FootPrintLibrary:LMH6574	1	LMH6574MAX/NOPB	\$4.48	\$4.48	
PCB	NA	NA	1	NA	NA	\$10.00	
Universal Remote	NA	NA	1	NA	NA	\$115.00	
Remote Receiver	NA	NA	1	NA	NA	\$50.00	
Xbee Pro	NA	NA	2	NA	NA	\$90.00	
Thermal Camera	NA	NA	1	NA	NA	\$50,000.00	
Sony Camera	NA	NA	1	NA	NA	\$32,000.00	
Backup Camera	NA	NA	1	NA	NA	\$35.00	
Front Drive Camera	NA	NA	1	NA	NA	\$35.00	
						\$82,379.52	Total Cost

Table 10.1: Project BOM

10.3: Engineering Logs

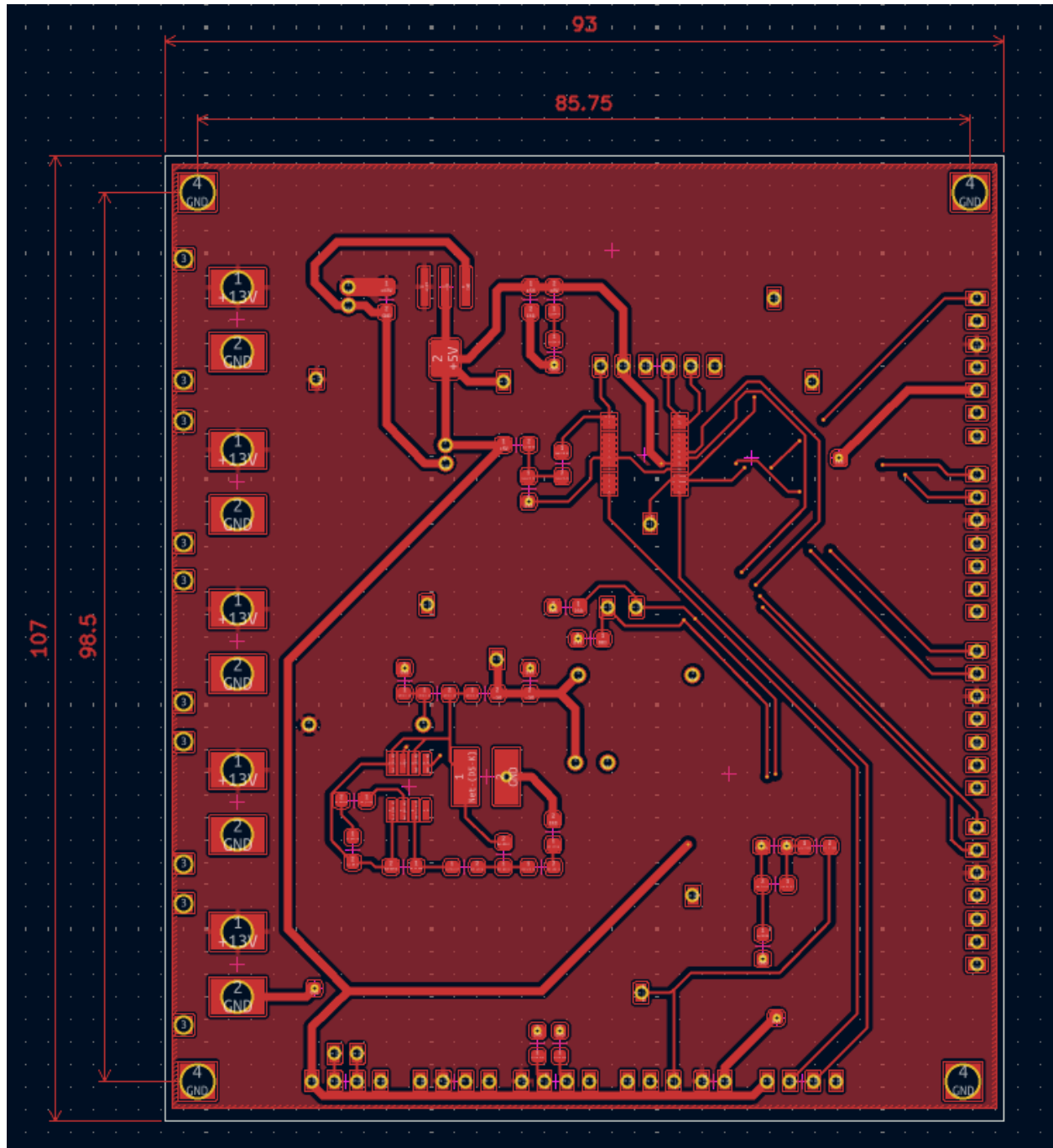
						Sum of Weeks		What was accomplished
March 9-13	9	10	11	12	13			
	6	7	6	7	6	32		Discovery/Previous Project Review
March 16-20	16	17	18	19	20			
	7	7	7	6	5	32		Discovery/Previous Project Review/ Coding
March 30-April 3	30	31	1	2	3			
	6	7	7	7	6	33		Coding
April 6-10	6	7	8	9	10			
	7.5	7	7	7	7.5	36		Coding/KiCad
April 13-17	13	14	15	16	17			
	7	7	7	6	5	32		KiCad
April 20-24	20	21	22	23	24			
	6	6.5	6	7	6.5	32		Mounting Camera/Cleaning up code
April 27-May 1	27	28	29	30	1			
	5.5	5.5	5.5	6	7	29.5		Mounting Camera/Report
May 4-8	4	5	6	7	8			
	6					6	Total @ \$50	Report/Presentation
Total						232.5	\$11,625.00	

Table 10.2: Rudy's Time Log

Date Range	Monday	Tuesday	Wednesday	Thursday	Friday	Weekly Hours	Accomplishments
April 13-17	<u>13</u>	<u>14</u>	<u>15</u>	<u>16</u>	<u>17</u>	2	Discussed objectives and expectations with project leads
	0	2	0	0	0		
April 20-24	<u>20</u>	<u>21</u>	<u>22</u>	<u>23</u>	<u>24</u>	15.75	Look for datasheets, code C# serial program
	5.5	5.25	5	0	0		
April 27-May 1	<u>27</u>	<u>28</u>	<u>29</u>	<u>30</u>	<u>1</u>	27.25	Coding/testing, look for datasheets
	5	5.25	5.75	5.25	6		
May 4-8	<u>4</u>	<u>5</u>	<u>6</u>	<u>7</u>	<u>8</u>	18	Final changes, presentations
	11	7	0	0	0		
				Grand Total:		63	\$ 3,150.00

Table 10.3: Jonas's Time Log

10.4: PCB

**Figure 10.5: Camera PCB Front Copper**

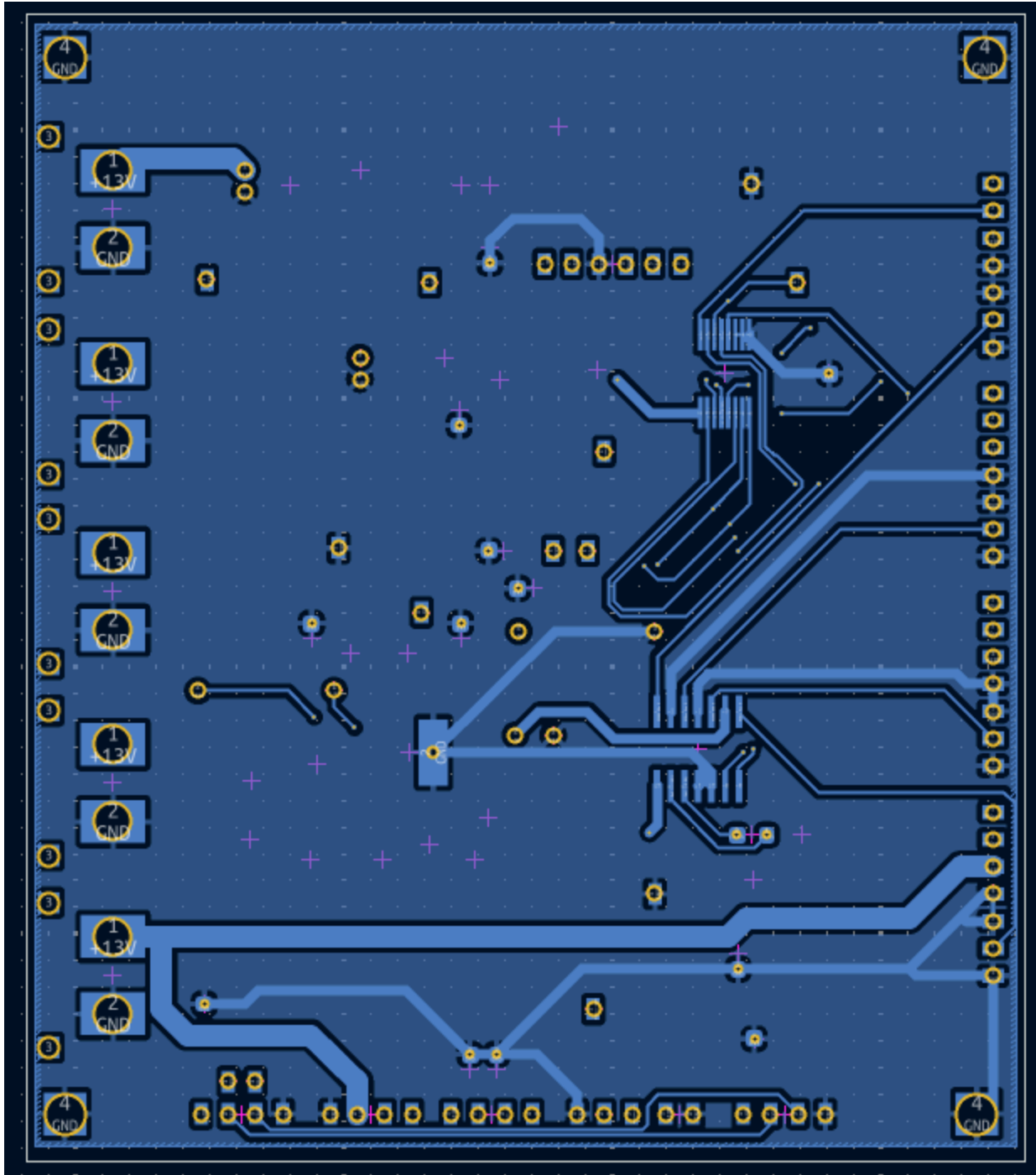


Figure 10.6: Back Copper

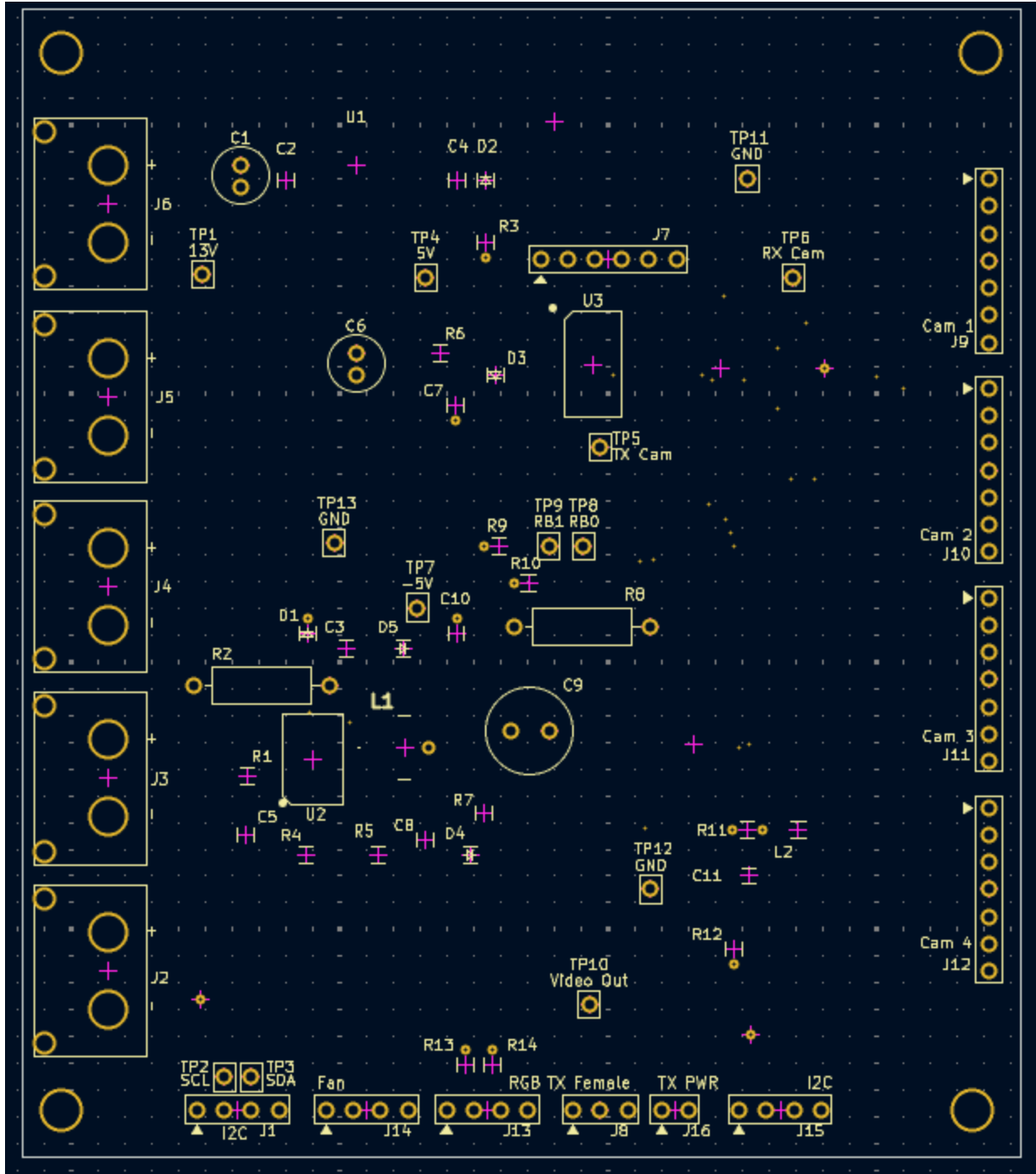


Figure 10.7: Front Silkscreen

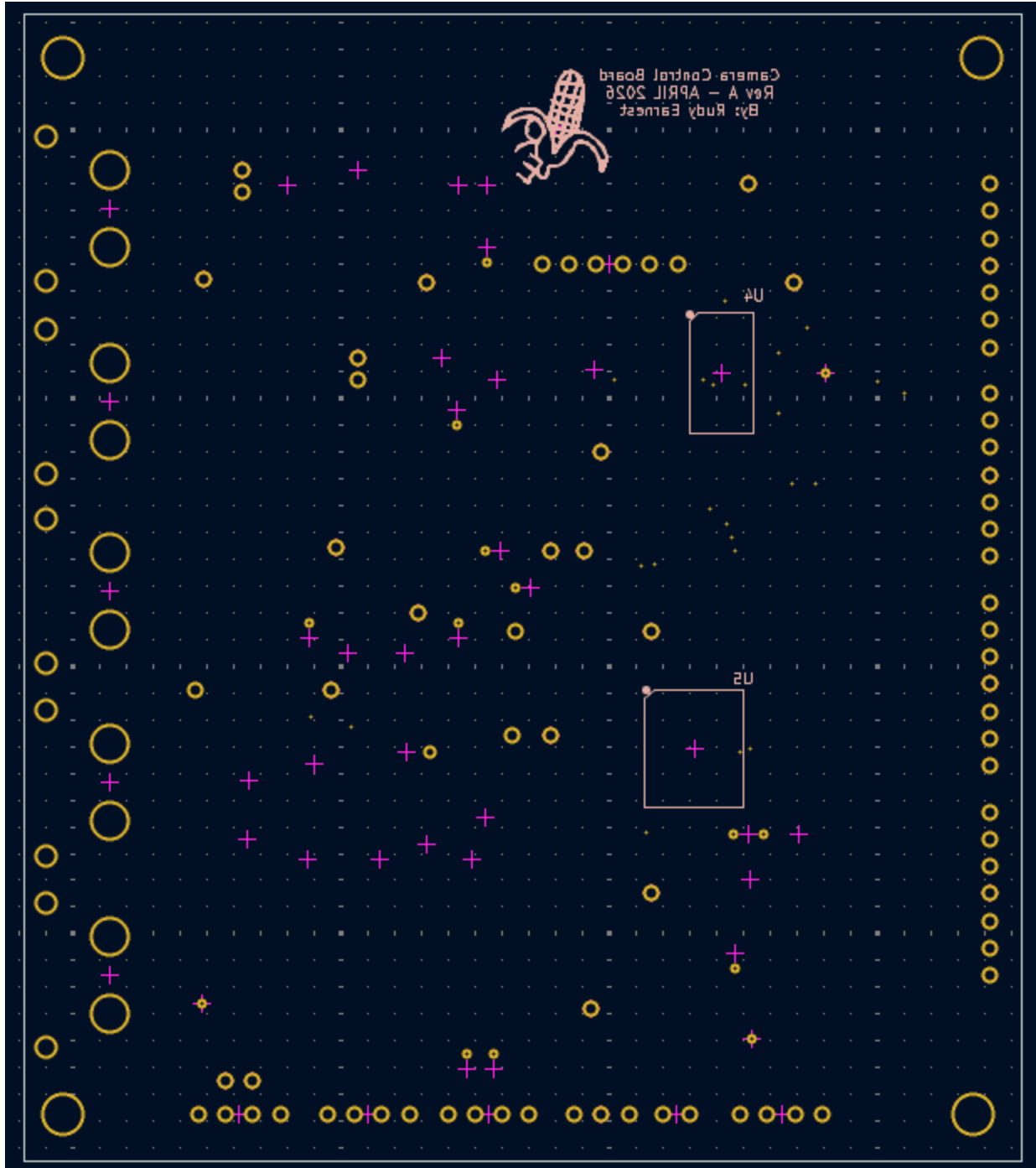


Figure 10.8: Back Silkscreen

10.5 Flowcharts

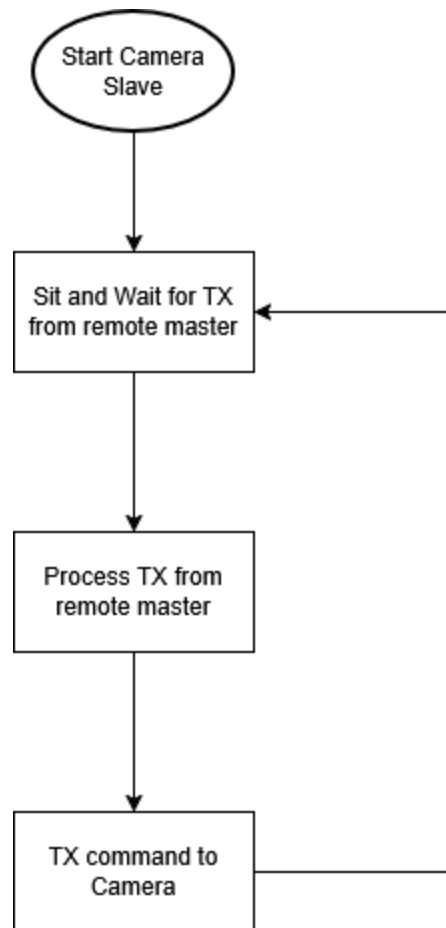


Figure 10.9: Camera Slave Program Overview Flowchart

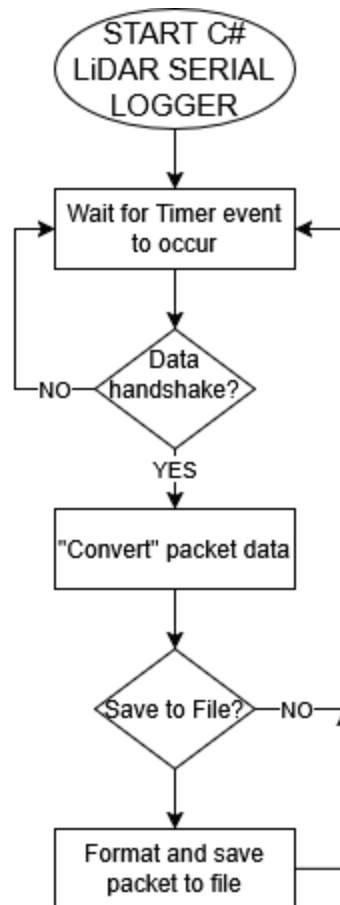


Figure 10.10: C# UART LiDAR Program Overview Flowchart

10.6: Github

<https://github.com/rc-earnest/BloodHound.git>



10.7: Schematics

{Page left blank for full page camera slave schematic}

{insert lidar schematics}

11: References

- Cruz Ulloa, Christyan, et al. “Autonomous Thermal Vision Robotic System for Victims Recognition in Search and Rescue Missions.” *Sensors (Basel, Switzerland)*, U.S. National Library of Medicine, 4 Nov. 2021, [pmc.ncbi.nlm.nih.gov/articles/PMC8588524/](https://pubmed.ncbi.nlm.nih.gov/articles/PMC8588524/).
- Lintvedt, Naomi. “Thermal Imaging in Robotics as a Privacy-Enhancing or Privacy-Invasive Measure? Misconceptions of Privacy When Using Thermal Cameras in Robots - Digital Society.” *SpringerLink*, Springer International Publishing, 6 Sept. 2023, link.springer.com/article/10.1007/s44206-023-00060-4.
- Quintin, Matthew Guariglia and Cooper. “Thermal Imaging Cameras Are Still Dangerous Dagnet Surveillance Cameras.” *Electronic Frontier Foundation*, 2 May 2020, www.eff.org/deeplinks/2020/04/thermal-imaging-cameras-are-still-dangerous-dagnet-surveillance-cameras.